Shelly Group 的产品与服务开放性解析

https://gemini.google.com/app/2e96d524224545f2

User prompt: Shelly group 的产品与服务有开放性吗？详细介绍其相关信息及开放生态

Response: **Shelly Group**（原名 Allterco Robotics，总部位于保加利亚的物联网与智能家居上市公司）的产品与服务在智能家居行业中以高度开放性（Openness）著称。

与许多将用户锁定在专属云端和封闭 App 中的传统品牌不同，Shelly 的核心设计理念是“开箱即用的高度开放”（Open by Design）。无论是普通消费者、智能家居极客，还是商业系统集成商（SI），都可以在不破坏保修、不强制拆机刷机的情况下，自由调用其底层能力。

以下是关于 Shelly 产品与服务开放性及其开放生态的详细介绍：

一、 核心开放性特征（技术与协议层面）

1. 100% 本地控制，零云端强制依赖（Local First）

- **独立运行**：Shelly 设备不需要绑定 Shelly Cloud 即可正常工作。用户完全可以关闭云连接，仅在局域网（LAN）内使用。
- **内置 Web 服务器（Embedded Web UI）**：每一台 Wi-Fi 类的 Shelly 设备内部都运行着一个轻量级 Web 服务器。用户只需在浏览器中输入设备的局域网 IP 地址，即可直接进入管理后台进行配置、固件更新、开关控制和自动化设置，无需下载任何 App。

2. 丰富的本地开放 API 与通讯协议

Shelly 原生固件（无需刷机）直接向用户和开发者开放了多种主流标准通信接口，且官方文档（Shelly Technical Documentation）极为详尽：

- **MQTT / MQTTs**：原生支持接入自建的 MQTT Broker（如 Mosquitto、EMQX），方便与各类开源平台或工业系统对接。
- **HTTP / REST API & RPC（远程过程调用）**：
  - 第 1 代产品（Gen1）采用标准的 HTTP RESTful 接口；
  - 第 2 代（Plus/Pro）、第 3 代（Gen3）产品全面升级为基于 JSON 的统一 **RPC 架构**，可通过 HTTP、WebSocket、MQTT 甚至串口调用相同的接口规范。
- **WebSockets**：支持双向全双工实时通信，状态反馈毫秒级响应。
- **Webhooks（动作钩子）**：设备可根据自身状态变化（如按键按下、温度超标、功率变化）主动向局域网或公网的其他设备发送 HTTP 请求，实现无中枢（Hub-less）的设备间直接联动。
- **CoIoT / mDNS**：用于局域网设备自动发现与状态广播。
- **Modbus TCP（Pro 系列专属）**：导轨安装的 Shelly Pro 系列原生支持工业与楼宇自动化常用的 Modbus TCP 协议，方便直接接入 PLC 或 BMS（楼宇管理系统）。

3. 设备端本地脚本引擎（Shelly Scripting）

从 Gen2（Plus/Pro 系列）和 Gen3 开始，Shelly 在设备固件中内置了基于 **mJS（受限版 JavaScript / ECMAScript 6 子集）** 的脚本引擎：

- **边缘计算**：用户可以直接在单个开关模块上编写并运行 JS 代码，无需外部服务器即可实现复杂的逻辑判断（例如：根据实时电价 API 自动控制热水器、读取第三方传感器数据、与其他品牌的局域网设备通信）。
- **多脚本并行**：支持在设备本地同时运行多个守护脚本。

二、 硬件层面的开放与极客友好度

1. **预留调试与刷机接口**：绝大多数 Shelly 模组（特别是基于 ESP8266 和 ESP32/ESP-Shelly 芯片的产品）在外壳上都直接留有 **串口调试针脚孔（Programming/Debug Header）**。用户无需暴力拆解外壳即可连接 USB-TTL 烧录器。
2. **全面兼容第三方开源固件**：虽然原厂固件已经足够开放，但如果用户想彻底统一自家的固件生态，Shelly 设备可以非常轻松地刷入 **ESPHome**、**Tasmota**、**Mongoose OS**（可刷写为原生 Apple HomeKit 固件）等开源固件。许多硬件型号在 ESPHome 和 Tasmota 官方库中都有现成的配置文件模板。
3. **Shelly Add-on 扩展接口**：支持通过专用硬件接口外接 DS18B20 温湿度传感器、模拟量输入（0-10V）、干接点输入等低成本标准传感器元件。

三、 广泛的开放生态与多技术路线布局

1. 智能家居开源平台“一等公民”

- **Home Assistant（HA）**：Shelly 与 Home Assistant 母公司 Nabu Casa 保持着极紧密的官方合作。HA 内置了经过最高等级认证（Platinum 铂金级）的原生 Shelly 集成，支持局域网自动发现、纯本地推送（Local Push），响应极快且极其稳定。此外，带蓝牙功能的 Shelly 设备还可以直接作为 Home Assistant 的 **ESPHome 蓝牙代理（Bluetooth Proxy）**，扩展全屋蓝牙覆盖。
- **其他开源平台**：原生完美支持 **Homebridge**、**openHAB**、**ioBroker**、**Domoticz** 以及 **Node-RED**（有专门的节点库）。

2. 多无线通信标准共存（不捆绑单一协议）

Shelly 没有将用户限制在单一无线技术上，而是全面拥抱行业开放标准：

- **Wi-Fi & 蓝牙（BLE）**：主力产品线（Gen1/Plus/Pro/Gen3），其中蓝牙不仅用于配网，还开放了 **BTHome 开放标准协议**（Shelly BLU 蓝牙传感器系列采用此开源广播协议，可直接被 HA 等平台免网关接收）。
- **Z-Wave（Shelly Wave 系列）**：收购知名 Z-Wave 厂商 Qubino 后推出的产品线，采用标准的 Z-Wave 800 系列芯片，兼容 SmartThings、Fibaro、Homey、HA Z-Wave JS 等所有标准 Z-Wave 网关。
- **Matter（Gen3 及后续产品）**：搭载自研 ESP-Shelly-C38F 芯片（8MB 闪存）的 Gen3 设备及部分 Pro 设备支持标准的 **Matter over Wi-Fi**，可直接跨生态接入 Apple Home、Google Home、Amazon Alexa 和 Samsung SmartThings。
- **KNX**：部分 Pro 系列产品提供原生 KNX 支持，打通传统高端有线智能家居生态。

3. 商业与云端生态开放（Shelly Cloud API）

如果用户或企业选择使用官方的 Shelly Smart Control 云服务，其云端同样开放了 **Cloud HTTP API** 和 **OAuth 接口**，允许第三方服务商、能源管理公司（EMS）或物业管理系统在获得授权后批量读取能耗数据或远程控制设备。

四、 总结：Shelly 开放性的核心优势与适用场景

维度	传统封闭智能家居品牌（如部分涂鸦定制、私有生态）	Shelly Group 开放生态
断网可用性	依赖云端，外网中断可能导致自动化失效	**100% 纯本地运行**，拔掉外网线依然正常联动
接口获取门槛	需申请开发者账号、审批、甚至按调用次数收费	**免申请、免费**，局域网内直接调用文档公开的 API
自定义逻辑	仅限 App 提供的简单“如果…就…”条件	支持 **本地 JavaScript 脚本**，可写任意复杂逻辑
硬件可玩性	隐藏焊点、加密锁死固件	预留串口插孔，**支持刷入 ESPHome / Tasmota**
厂商倒闭风险	服务器关停即“变砖”	只要硬件不坏，可永久在局域网或开源平台中使用

一句话总结： Shelly Group 的产品是目前市面上**对开发者、开源社区（如 Home Assistant）和系统集成商最为开放的商业级智能硬件之一**。它既提供了成熟稳定的商业级硬件认证（CE/UL等）和官方 App 体验，又保留了彻底开放的底层接口、本地脚本和硬件可编程能力。

User prompt: 详细介绍以下内容 “第 2 代（Plus/Pro）、第 3 代（Gen3）产品全面升级为基于 JSON 的统一 RPC 架构，可通过 HTTP、WebSocket、MQTT 甚至串口调用相同的接口规范。”

Response: 在 Shelly 从第 1 代（Gen1）向第 2 代（Gen2，即 Plus/Pro 系列）及第 3 代（Gen3）演进的过程中，底层固件经历了一次彻底的重构。其中最核心的改变，就是抛弃了 Gen1 时代零散的传统接口，全面转向了**基于 JSON-RPC 2.0 标准的统一远程过程调用（RPC）架构**。

这一架构的核心设计哲学是：**“协议与传输解耦（Transport-Agnostic）”**——无论你用什么通信通道连接设备，你发送的指令格式、调用的方法名称、返回的数据结构都是**完全一致**的。

以下为您深入剖析这一架构的技术细节与实际运作方式：

一、 为什么说它是“统一的”？（对比 Gen1 与 Gen2/Gen3）

在 **Gen1（第一代）** 产品中，HTTP 接口和 MQTT 话题（Topic）是两套完全割裂的逻辑：

- 用 HTTP 控制继电器：`http://<IP>/relay/0?turn=on`（URL 传参）
- 用 MQTT 控制继电器：向话题 `shellies/<device-id>/relay/0/command` 发送纯文本 `on`。 开发者如果把系统从 HTTP 迁移到 MQTT，必须重写所有的接口解析代码。

而在 **Gen2 / Gen3** 中，Shelly 将设备内部的所有功能模块化为**组件（Components）**，如 `Switch`（开关）、`Input`（输入）、`Light`（灯光）、`WiFi`、`Sys`（系统）、`Script`（脚本）等。每个组件都拥有标准的方法（如 `.Set`、`.GetStatus`、`.GetConfig`）。 无论通过哪种方式通信，调用的永远是同一个方法名（例如 `Switch.Set`）和同一套 JSON 参数。

二、 核心数据报文格式（JSON-RPC 2.0）

Shelly 的 RPC 严格遵循（并略微扩展了）标准的 JSON-RPC 2.0 规范。每次交互都包含**请求帧（Request Frame）和响应帧（Response Frame）**。

1. 标准请求报文（Request）

假设我们要打开编号为 0 的继电器开关：

```
{
  "id": 101,
  "src": "my_home_server",
  "method": "Switch.Set",
  "params": {
    "id": 0,
    "on": true
  }
}
```

- `id`：请求标识符（数字或字符串），设备响应时会原样带回，方便异步通信时将“响应”与“请求”对应起来。
- `src`（来源）：调用方自定义的名称（在 MQTT 和 WebSocket 中非常关键，用于告诉设备把响应发回给谁）。
- `method`：`<组件名>.<动作名>`，如 `Switch.Set`、`Sys.Reboot`、`Shelly.GetDeviceInfo`。
- `params`：传入给该方法的 JSON 对象参数。

2. 标准响应报文（Response）

设备执行成功后，会返回如下结构的 JSON：

```
{
  "id": 101,
  "src": "shellyplus1pm-a8032abe54",
  "dst": "my_home_server",
  "result": {
    "was_on": false
  }
}
```

- `src`：变成了设备自己的 ID。
- `dst`（目的地）：对应请求中的 `src`。
- `result`：返回的结果数据（在此例中，它不仅打开了开关，还贴心地返回了操作前开关的状态 `"was_on": false`）。如果执行出错，则会返回 `"error": {"code": ..., "message": ...}`。

三、 四大传输通道（Transports）如何调用同一套接口?

这就是“通过 HTTP、WebSocket、MQTT 甚至串口调用相同的接口规范”的具体实现方式：

1. 通过 HTTP 调用（适合单次控制、脚本调用、Webhook）

HTTP 提供了两种调用方式，底层指向同一个 RPC 方法：

- **HTTP POST（纯正 RPC 方式）**： 向统一入口 `http://<设备IP>/rpc` 发送 POST 请求，Body 直接放上面的标准 JSON 请求报文。
- **HTTP GET（快捷 REST 风格映射）**： 为了方便用户在浏览器地址栏或简单 Webhook 中直接调用，Shelly 允许把 RPC 方法直接映射为 URL： `http://<设备IP>/rpc/Switch.Set?id=0&on=true` *(注：这只是语法糖，设备内部依然会将其转化为 `Switch.Set` 的 RPC 调用并返回相同的 JSON 结果)*。

2. 通过 WebSocket 调用（适合局域网实时中枢，如 Home Assistant）

- **连接地址**：`ws://<设备IP>/rpc`
- **工作机制**：建立长连接后，客户端直接向 WebSocket 通道发送上述 JSON 请求文本，设备立即在通道内回复 JSON 响应。
- **核心优势（主动推送通知 Notification）**：一旦连接建立，即使你不发请求，当有人按下了墙上的物理面板、或者功率发生变化时，Shelly 会**主动推送**名为 `NotifyStatus` 或 `NotifyEvent` 的 RPC 帧给客户端，实现**零延迟的纯本地状态同步**。

3. 通过 MQTT 调用（适合跨网段、物联网平台、大规模部署）

在 MQTT 中没有直接的“请求-响应连接”，但 Shelly 巧妙地利用 `src` 字段在 MQTT 上实现了双向 RPC：

1. **发送请求**：客户端向话题 `<shelly-id>/rpc` 发布（Publish）刚才那段 JSON 报文（其中包含 `"src": "my_client"`）。
2. **接收响应**：设备处理完后，会自动读取请求里的 `src`，然后向话题 `my_client/rpc` 发布响应的 JSON 报文！ 这意味着你在 MQTT 里不仅能发控制指令，还能用 MQTT 远程修改设备的 Wi-Fi 密码、读取配置、甚至上传本地脚本，功能与局域网 HTTP 100% 对等。

4. 通过串口（UART / Serial）调用（适合工业集成、单片机硬改、无网环境）

这是极客和工业用户非常喜欢的功能：

- 通过设备上的硬件调试接口（UART 针脚）或连接 USB 设备，将波特率设为 `115200`（8N1）。
- 直接通过串口发送一行刚才的 JSON 字符串并加上换行符，设备就会通过串口吐出 JSON 响应。
- **应用场景**：即使在完全没有 Wi-Fi、蓝牙和以太网的严苛工业环境，或者用一块外部的 Arduino / ESP32 / 树莓派通过两根杜邦线直接连上 Shelly，也能用完全相同的 JSON API 控制它并读取电能计量数据。

*(补充：其实还有第 5 种通道——**BLE 蓝牙 GATT** 以及第 6 种——**设备内部 mJS 脚本** `Shelly.call("Switch.Set", ...)`，它们调用的也是完全相同的 RPC 接口)。*

四、 这种统一 RPC 架构带来的巨大优势

1. **Web 后台与 API 100% 同源（Dogfooding）**： 当你用浏览器打开一台 Gen2/Gen3 Shelly 设备的内置网页后台（Web UI）时，这个网页本身并没有任何特殊后门，它就是通过浏览器的 WebSocket 连接到设备的 `/rpc` 接口来加载界面和保存设置的。**这意味着：只要是你在官方网页后台能点出来的任何功能、能看到的任何数据，你都能 100% 通过公开的 RPC API 用代码实现。**
2. **一次开发，到处运行**： 开发者编写一套 JSON 报文生成与解析逻辑，就可以同时用于本地 HTTP 轮询、WebSocket 实时监听、或者云端 MQTT 远程管理，极大地降低了与第三方系统（如 Node-RED、自研 EMS 能源管理系统、楼宇中控）的对接成本。
3. **内置内省机制（Introspection）**： 你甚至不需要查文档，直接向任何一台设备调用 `Shelly.ListMethods`，设备就会以 JSON 列表的形式告诉你：“我这台硬件目前支持哪些组件、哪些 RPC 方法”。这对于自动化插件开发极为友好。

User prompt: 有没有比上面的方案更合适更好的方法

Response: Shelly 原生的 **统一 JSON-RPC 架构** 虽然在“开箱即用”的商业固件中已经做得非常出色，但它本质上是为了**兼顾普通用户、网页后台、云端和开发者**而设计的“通用型方案”。

在不同的实际应用场景下，直接手写代码去对接 Shelly 的原生 JSON-RPC **往往不是最高效、最稳定或性能最好的方法**。

根据你的具体目标（是做智能家居集成、工业控制、极客深度定制，还是跨品牌互联），以下有 **5 种比直接调用原生 JSON-RPC 更合适、更好的替代方案**：

方案一：如果你是为了接入智能家居/自动化平台

👉 更好的方法：使用成熟平台的原生集成 SDK（如 `aioshelly`）或 Node-RED 节点，而非手写 RPC 报文

自己去维护 WebSocket 的心跳保活、断线重连，或者在 MQTT 里处理 `src`/`dst` 异步匹配是非常繁琐的。

- **做法**：
  - **Home Assistant 原生集成**：直接使用 HA 官方内置的 Shelly 插件（底层基于 Python 库 `aioshelly`）。它虽然底层走的也是 WebSocket RPC，但已经为你封装好了自动发现（mDNS）、连接池管理、状态同步和异常恢复。
  - **MQTT 简化话题（MQTT Control / Status Topics）**：如果你觉得在 MQTT 里拼装 JSON-RPC 太重，Shelly Gen2/Gen3 实は保留了**简化的 MQTT 状态与控制话题**（需在设置中勾选 `Generic status update over MQTT`）。开启后，你可以直接向 `<shelly-id>/command/switch:0` 发送简单的字符串 `on` 或 `off`，而不需要构造庞大的 JSON-RPC 结构体。

方案二：如果你追求极致的统一管理、高安全性与代码即配置

👉 更好的方法：直接刷入 ESPHome 开源固件（抛弃原厂固件与 JSON-RPC）

JSON-RPC 是基于纯文本（JSON 字符串）解析的，占用内存较多，且当你家里有 50 台设备时，你需要逐台去配置或写脚本维护。

- **为什么 ESPHome 更好**：
  1. **Protobuf 二进制通信**：ESPHome 使用基于 Google Protocol Buffers 的 Native API 与中枢（如 Home Assistant）通信，**二进制流比 JSON 文本解析体积小得多、速度更快、CPU 和内存开销极低**。

2. **原生加密（Noise Encryption）**：Shelly 的局域网 HTTP/WebSocket 默认是明文的（只支持 Digest Auth 密码校验，配置局域网 TLS 证书非常麻烦）；而 ESPHome 原生支持端到端加密通信。
3. **配置即代码（YAML 集中管理）**：你只需要在一个面板里维护几行 YAML 代码，就能一键编译并 OTA 批量更新家里所有 Shelly 设备的逻辑，彻底告别逐个 IP 登录网页后台或管理零散的 mJS 脚本。

方案三：如果你是做工业自动化、PLC 对接或楼宇自控（BMS）

👉 更好的方法：使用 Modbus TCP 或 KNX（Shelly Pro 系列原生支持）

在工业场景中，让西门子/施耐德的 PLC（可编程逻辑控制器）或组态软件去解析 JSON 字符串是一场灾难。

- **为什么 Modbus TCP / KNX 更好**：
  1. **工业标准寄存器直接读写**：Shelly Pro 系列（导轨式）原生内置了 Modbus TCP Server。工业上位机不需要懂任何 JSON，直接读取标准的保持寄存器（Holding Registers）就能拿到电压、电流、功率数据，写入线圈（Coils）就能控制继电器吸合。
  2. **高确定性与抗干扰**：相比应用层复杂的 JSON 文本解析，底层的二进制字节流协议在工业高频轮询下更加稳定可靠。

方案四：如果你要求两个设备之间极速响应、绝不依赖路由器/服务器

👉 更好的方法：使用 设备端本地 mJS 脚本 + Shelly BLU（BTHome 蓝牙广播） 或 直接 Webhook

如果通过外部服务器用 JSON-RPC 监听按键再下发开灯指令，路径是：开关 -> 路由器 -> 服务器 -> 路由器 -> 电灯，一旦路由器重启或卡顿就会失灵。

- **为什么这种方式更好**：
  1. **蓝牙直连联动（脱离 Wi-Fi）**：你可以买一个 Shelly BLU 蓝牙无线按钮，配合墙内的 Shelly Gen2/Gen3 开关模块。模块内置的蓝牙网关可以直接在芯片底层捕获按钮的 **BTHome 广播包**，在设备内部毫秒级触发继电器动作，**就算家里路由器断电、Wi-Fi 彻底瘫痪，无线开关依然能秒开灯**。
  2. **设备端闭环自治**：把控制逻辑直接写进 Shelly 内部的 mJS 脚本里（例如：检测到功率过载或温度过高，内部直接切断输出并联动蜂鸣器），完全省去外部 RPC 通信的开销与故障点。

方案五：如果你追求跨品牌、跨大厂生态的统一标准

👉 更好的方法：使用 Matter 协议 或 Z-Wave（Shelly Wave 系列）

Shelly 的 JSON-RPC 终究是 **Shelly 自己定义的专有接口规范**——你为 Shelly 写的 RPC 代码，换成一个 Sonoff、Aqara 或飞利浦的开关就完全不能用了。

- **为什么 Matter / Z-Wave 更好**：
  1. **Matter（Gen3 原生支持）**：采用标准化的 IPv6 局域网集群模型（Cluster Data Model）。通过 Matter 接入后，无论是 Apple Home、Google Home 还是 Home Assistant，都用全球统一的标准指令控制它，无需针对 Shelly 编写任何特定适配代码，未来替换其他品牌的硬件也无需改动系统。
  2. **Z-Wave（Shelly Wave 系列）**：如果你担心几十个 Wi-Fi 设备拖垮家用路由器，基于 Sub-1GHz 频段的 Z-Wave 组网在穿墙能力、低功耗和信道抗干扰上，远优于基于 Wi-Fi 的 JSON-RPC 方案。

总结对比：你该如何选择？

你的核心诉求 / 场景	是否推荐原生 JSON-RPC?	更合适、更好的替代方案
普通智能家居玩家 / 快速接入 HA	❌ 没必要手写	**HA 官方 Shelly 集成** 或 **Matter 协议**
极客玩家 / 几十台设备统一维护	❌ 维护成本高	刷入 **ESPHome**（二进制 API + YAML 集中管理）
老旧系统 / 简单 MQTT 脚本对接	❌ JSON 太啰嗦	开启 Shelly 的 **MQTT 简单状态与控制话题**（纯文本 on/off）
工业控制 / PLC / 能源管理系统	❌ PLC 难解析 JSON	选用 Shelly Pro 系列，直接走 **Modbus TCP** 或 **KNX**
追求极致稳定 / 怕 Wi-Fi 路由器挂掉	❌ 依赖网络层	**Shelly Wave (Z-Wave)** 或 **蓝牙 BTHome + 本地脚本**
开发第三方商业 App / 深度定制网页	✅ **非常推荐**	**原生 JSON-RPC (WebSocket / HTTP)** 是此场景的最佳选择

User prompt: 详细总结shelly group产品定义与技术选型的得失

Response: 如果站在智能硬件产品经理（PM）与物联网系统架构师（Architect）的双重视角来审视 **Shelly Group**，它的发展史其实是一部非常经典的“用极客口碑撬动大众与商业市场，同时在性能、成本与协议演进中不断做技术妥协与重构”的教科书。

以下是对 Shelly Group 在**产品定义**与**技术选型**两个维度的“得（核心优势与红利）”与“失（历史包袱与架构妥协）”的深度总结：

一、 产品定义（Product Definition）的“得”与“失”

✅ 产品之“得”：精准的切入点与三维生态破圈

1. **精准切入“隐形后装改造（Retrofit）”与“零网关门槛”**
   - **保留原有习惯**：早期智能家居市场要么是昂贵的全屋前装（如 KNX），要么需要换掉全屋开关面板并购买专用网关（如 Zigbee 体系）。Shelly 选择做**隐藏在传统墙壁开关暗盒（Wall Box）背后的微型模组**，不改变家庭原有的装修风格和物理按键习惯。
   - **单品即用（No Hub Required）**：依托家家户户现成的 Wi-Fi 路由器，用户只需花十几欧元买一个开关就能跑通全套自动化，将用户的“首单决策门槛”降到了极致。
2. **“小白 + 极客 + 专业电工”三位一体的罕见兼容定位**
   - **用极客做免费宣发**：大多数厂商防着极客刷机，Shelly 却主动把本地 API、编程接口甚至芯片调试针脚开放给极客和 Home Assistant 社区。这群高粘性技术爱好者成为了 Shelly 全球扩张最强大的免费口碑推手和义务测试员。
   - **向高毛利 B 端配电箱市场跃迁（Pro 与 CB 系列）**：在积累了 C 端口碑后，Shelly 迅速推出导轨式（DIN Rail）的 **Pro 系列**及智能断路器（PRO CB），增加有线以太网口（解决金属配电箱屏蔽 Wi-Fi 信号痛点）和 Modbus/KNX 支持，成功切入专业电工、商业楼宇和能源管理（EMS）市场。
3. **克制的商业化与隐私信任（Local-First）**
   - 在欧美对数据隐私（GDPR）极度敏感的市场，Shelly 坚持“不强制上云、断网可用”，靠硬件规模效应和可选的高级能源看板（Premium）盈利，建立了极高的品牌安全感。

❌ 产品之“失”：安装门槛限制与 SKU 认知过载

1. **强电接线门槛（Mains Wiring Barrier）制约了向纯大众市场的破圈**
   - Shelly 的核心产品几乎全部涉及 110V–240V 强电改造。对于懂技术的 DIY 玩家来说接几根线很容易，但对于占人口大多数的普通消费者而言，自己动强电存在触电和火灾恐惧，而在欧美聘请持证电工上门安装一个十几欧元的模块，人工费往往是硬件本身的数倍。这导致 Shelly 在从“极客圈”迈向“纯大众消费级市场”时遇到了天然的增长摩擦力。
2. **产品线迭代过快导致的 SKU 极度膨胀与命名混乱**
   - 目前市场上同时在售 **Gen1、Plus（Gen2）、Pro、Mini、Gen3、Gen4、Wave（Z-Wave）、BLU（蓝牙）** 等多个系列。
   - **定位与代际错位**：例如“Pro”代表的是导轨安装形态，而非芯片代际——市面上绝大多数在售的 Pro 设备内部依然运行着老旧的 Gen2 硬件，并不支持最新的 Matter 或 Zigbee（仅少数新型号如 Pro 1PM 40A Gen4 例外）。普通买家在选购时往往需要查阅复杂的对比表才能搞清功能差异。
3. **“重幕后、轻台前”，缺乏高感知的消费级交互终端**
   - Shelly 长期专注于暗盒与配电箱内的“隐形控制与计量”，在大众更容易感知的消费级终端品类（如高品质智能灯具、智能门锁、全屋环境家电）上布局较弱，品牌在非技术人群中的心智知名度不如 Philips Hue 或 Aqara。

二、 技术选型（Technology Selection）的“得”与“失”

✅ 技术之“得”：软硬解耦与极致的边缘计算压榨

1. **深度绑定乐鑫（Espressif）生态并走向芯片联合定制**
   - 从早期的 ESP8266，到 Gen2 的 ESP32，再到 Gen3 联合定制具备 8MB 大闪存的 **ESP-Shelly-C38F** 芯片，直至 Gen4 升级为支持 Wi-Fi 6 + 蓝牙 5 + Zigbee 的多模芯片。Shelly 吃透了 RISC-V / Xtensa 架构的高性价比红利，用极低的 BOM（物料成本）在单颗微控制器上塞进了媲美轻量级网关的算力。
2. **“设备即微型服务器”的边缘软件架构（统一 RPC + mJS 脚本）**
   - 通过全功能内置 Web UI、统一 JSON-RPC 2.0 架构以及设备端 mJS 脚本引擎，Shelly 将传统架构中属于“云端或中枢网关”的逻辑下沉到了终端节点。这不仅省去了用户的网关成本，还让设备具备了极强的独立自治与跨协议桥接能力。
3. **“借鸡生蛋”的蓝牙网关策略（BLE Gateway + BTHome）**

- Wi-Fi 功耗太高，无法做纽扣电池供电的门磁或温湿度计。Shelly 的解法极其巧妙：**让家里已经接在强电上的每一台 Gen2/Gen3/Gen4 Wi-Fi 开关直接兼任“蓝牙网关”**，并采用开源的 **BTHome** 协议接收自家 Shelly BLU 低功耗传感器的广播。既补齐了低功耗传感器短板，又实现了零网关硬件成本的全屋蓝牙覆盖。

❌ **技术之“失”：Wi-Fi 物理瓶颈与多协议融合的割裂**

1. **早期“All-in Wi-Fi”带来的发热、功耗与单火线痛点**
   - **散热与体积的物理矛盾**：Wi-Fi 射频维持长连接的功耗远高于 Zigbee。在通风极差、塞满电线的狭小暗盒内，Wi-Fi 芯片发热叠加继电器大电流发热，导致早期紧凑型产品容易面临高温降频或元器件寿命衰减。
   - **“单火线（No-Neutral）”体验滞后**：在许多老房子的开关盒里没有零线（Neutral Wire）。Zigbee 设备因为微安级的极低功耗可以轻松实现单火线稳定运行，而 Wi-Fi 设备瞬时电流极大，导致 Shelly 的单火线产品（如 1L/2L 系列）往往需要额外并联旁路电容（Bypass），且在新一代硬件上的迭代速度明显滞后于零火线版本。
   - **家用路由器承载压力**：当全屋部署超过 50 个以上的 Wi-Fi 开关时，对普通家用路由器的带机量、mDNS 广播风暴处理能力提出了较高要求。
2. **重型软件栈带来的硬件资源内耗（Flash 与 RAM 焦虑）**
   - 相比于二进制协议（如 ESPHome Native API 或 Zigbee Cluster），在单片机上同时跑内置网页服务器、JSON 文本解析、TLS 加密证书栈、mJS 脚本虚拟机和 WebSocket 是非常奢侈的。
   - 这种“重型软件栈”直接导致 **Gen2（4MB 闪存）在面对庞大的 Matter 协议栈时捉襟见肘**，无力支持 Matter；迫使 Shelly 不得不在短短一两年内紧急推出 **Gen3（扩容至 8MB 闪存）** 来为 Matter 腾出空间。
3. **Gen4 多协议架构的“鱼与熊掌不可兼得”（Wi-Fi 与 Zigbee 互斥）**
   - 为了弥补 Wi-Fi 的短板，最新一代 **Gen4** 终于在硬件上集成了 Zigbee 射频与 Wi-Fi 6。
   - **架构妥协**：在 Gen4 上，Zigbee 并不是与 Wi-Fi 并行运行的第二通道，而是**需要切换的替代固件（Alternative Firmware）**。一旦用户将 Gen4 切换为 Zigbee 模式接入网关，设备就会彻底断开 Wi-Fi 并失去 IP 地址——这意味着 Shelly 最引以为傲的 **内置 Web 界面、MQTT、Matter、HTTP RPC 在 Zigbee 模式下将全部失效**，甚至连固件升级都必须先切回 Wi-Fi 模式才能进行。

三、 全景总结对比表

评估维度	核心决策	收获的红利（“得”）	付出的代价（“失”）
产品形态	暗盒后装模组 + 导轨模组	免换面板、隐藏式改造；顺利打通 C 端 DIY 与 B 端电工配电箱市场	强电安装门槛高，阻碍非技术大众消费者自装普及
产品矩阵	Gen1~Gen4 + Plus/Pro/Wave/BLU	覆盖几乎所有细分电压、功率与安装场景	迭代过密、SKU 繁杂，Pro 系列与芯片代际不同步，选购心智负担重
主通信协议	以 Wi-Fi 为绝对核心（后补齐 Z-Wave/Zigbee）	免网关（No Hub），单品购买门槛极低，带宽充裕	暗盒散热压力大、单火线（无零线）方案难做、多设备考验路由器性能
软件与接口	统一 JSON-RPC + 内置 Web + 本地 mJS 脚本	极客与系统集成商体验封神，完全脱离云端也能实现复杂边缘自治	内存与闪存开销巨大，导致 4MB 的 Gen2 无法升级支持 Matter
多协议融合	Gen4 引入 Wi-Fi 6 + Zigbee + Matter	终于实现“一颗硬件通吃主流无线生态”	Wi-Fi 与 Zigbee 互斥运行，开启 Zigbee 后丧失全部 IP 级高阶能力

Sources

1. Shelly Group: Redefining Smart Home Automation One Device
2. Shelly Gen 3 vs Gen 4 vs Pro: what actually differs | Synced

---

User prompt: 如果我们要设计一款新的智能家居或物联网硬件产品，该如何借鉴 Shelly Group 的经验并规避它的技术短板?

Response: 设计一款全新的智能家居或物联网硬件产品时，Shelly Group 是极佳的参考样本——它证明了“无云端强绑定 + 极客生态口碑 + 软硬解耦”可以让一家硬件公司在全球巨头夹击下异军突起；但同时，它走过的弯路（如 Wi-Fi 发热、单火线痛点、Flash 闪存焦虑、多协议割裂、SKU 爆炸）也是新入局者最宝贵的“避坑指南”。

如果你今天要从零定义一款极具竞争力的物联网/智能家居硬件，可以从**通信与硬件底座、嵌入式软件架构、产品形态与 SKU 策略、生态冷启动**四个维度，采取以下“取其精华、补其短板”的架构方案：

一、 通信与硬件底座：用“原生 IP 低功耗网状网”破解 Wi-Fi 与 Zigbee 的两难

Shelly 最大的技术包袱在于早期过度依赖 Wi-Fi（导致暗盒发热大、无零线/单火线难做），而在 Gen4 引入 Zigbee 后，又因为 Zigbee 不是 IP 协议，导致切换到 Zigbee 模式后彻底丧失了引以为傲的 Web UI、MQTT 和 HTTP RPC 能力。

1. 借鉴 Shelly：

- **拥抱 RISC-V 多模芯片与预留调试口**：继续选用乐鑫（如 ESP32-C6 / ESP32-H2）或 Silicon Labs（EFR32MG24）等高性价比多模 SoC；在外壳上保留免拆机的串口/SWD 调试孔，向极客传达“硬件归用户所有”的态度。
- **强电设备兼任低功耗网关**：所有接市电（AC）的节点，默认开启蓝牙（BLE）扫描代理与路由中继能力。

2. 规避短板（更优解）：

- **用 Thread（IPv6）替代“Wi-Fi 或 Zigbee 二选一”**：
  - 为什么 Shelly 切到 Zigbee 会丢掉所有 IP 接口？因为 Zigbee 没有 IP 层。
  - **破局方案**：对于功耗敏感、空间狭小（如暗盒开关、单火线开关、传感器）的产品，主打 **Matter over Thread**。Thread 底层与 Zigbee 一样采用极低功耗的 **IEEE 802.15.4 射频**（彻底解决暗盒发热与单火线取电难的问题，待机功耗仅毫瓦级），但它的网络层是**原生 IPv6**！
  - 这意味着：只要家里有边界路由器（如 Apple TV、HomePod、带 Thread 的路由器），你的设备在保持微安级低功耗的同时，**依然拥有独立的局域网 IPv6 地址**，依然可以直接通过 UDP/CoAP 甚至轻量级 TCP 暴露本地 RPC 接口，真正做到“低功耗与全功能 IP 接口兼得”。
- **Flash 与内存一步到位，杜绝“半代升级”**：
  - 吸取 Shelly Gen2（4MB Flash）因装不下 Matter 协议栈而被迫紧急推出 Gen3（8MB Flash）的教训。
  - **硬件基线**：只要产品规划了双分区安全 OTA + Matter 协议栈 + 本地自动化引擎，**Flash 必须以 8MB 起步（高端网关类建议 16MB + 外部 PSRAM）**。几毛钱人民币的 BOM 成本增加，可以换来未来 5 年的固件生命周期。
- **硬件级过零检测（Zero-Crossing）与磁保持继电器**：
  - 针对暗盒高温痛点，在硬件上加入**过零检测电路**（仅在交流电电压过零点吸合继电器，防止浪涌电流烧蚀触点）并采用**磁保持继电器（Latching Relay）**——只在切换瞬间耗电，保持接通状态时线圈零功耗、零发热。

二、 嵌入式软件架构：保持“同源 RPC”，但实现“文本与二进制双轨制”

Shelly 的统一 JSON-RPC 2.0 架构极其优雅，但纯文本 JSON 解析和重型 mJS 引擎在单片机上非常吃内存，且默认明文传输缺乏安全性。

1. 借鉴 Shelly：

- **100% 本地优先（Local-First）与“Dogfooding（自己吃自己的狗粮）”**：不绑定云端即可激活全部功能；设备内置的 Web 管理页面必须 100% 调用公开的本地 API（不留私有后门），确保用户在网页上能点出来的任何设置，都能用 API 自动化实现。

2. 规避短板（更优解）：

- **设计“JSON + CBOR/Protobuf”双轨制统一 RPC**：
  - 底层数据模型依然采用统一的 `<Component>.<Method>` 架构。
  - **对外（面向人类开发者 / Webhook / 浏览器）**：暴露标准的 **JSON-RPC**（通过 HTTP / WebSocket / MQTT），保证极低的对接门槛。
  - **对内（面向高频状态同步 / Home Assistant 集成 / 蓝牙与 Thread 传输）**：同一套数据结构自动序列化为 **CBOR（简明二进制对象表示）或 Protobuf**，并配合 Noise 协议加密（类似 ESPHome Native API）。这样既保留了 Shelly 的易用性，又获得了媲美 ESPHome 的超低内存开销和端到端加密安全。
- **极简压缩的静态 Web UI（Single-File Gzip SPA）**：
  - 使用 Preact 或 Svelte 编写设备内置网页后台，将其打包为单个经过 Gzip 强压缩的静态文件（体积可控制在 **50KB~80KB** 以内）存入 Flash。浏览器访问设备时，单片机只需吐出这一个静态压缩包，所有渲染和逻辑全部交给用户的手机/电脑浏览器算力完成，彻底解放单片机 CPU。
- **用“结构化规则表 + 沙盒看门狗”优化本地脚本**：

- Shelly 的 mJS 脚本虽然强大，但普通用户写不出代码，而写错的死循环脚本容易导致设备内存溢出或卡死。
- **破局方案**：提供两层边缘能力——第一层是**基于 JSON 的本地状态机规则表（Local Rule Engine）**，让用户通过简单的条件配置就能实现 90% 的断网联动；第二层针对极客提供轻量级脚本（如 Berry 或受限 JS），但必须运行在带独立内存配额与硬件看门狗（Watchdog）的沙盒中：一旦脚本超时或引发连续重启，固件在启动前 3 秒内自动安全挂起脚本，绝不影响物理开关面板的正常救砖与照明。

三、 产品定义与形态：用“强弱电分离”解决安装恐惧与 SKU 地狱

Shelly 面临的最大大众化障碍是“普通人不敢接 220V 强电”，以及代际演进导致的几十上百个零散 SKU（Plus/Pro/Mini/Gen3/Gen4/1PM/2PM...）。

1. 借鉴 Shelly：

- **紧抓“电能计量（PM）”与“后装改造”刚需**：在电价高企和低碳趋势下，精准的功率、电压、电流监控依然是智能硬件的核心卖点；同时兼容标准导轨（DIN Rail）与传统暗盒。

2. 规避短板（更优解）：

- **采用“强电底座 + 弱电智能核心”的可插拔模块化架构**：
  - 将产品拆分为两部分：
    1. **强电底座（Power & Relay Base）**：包含 220V 接线端子、变压器、继电器和计量芯片，通过安规认证（UL/CE/CCC）。一生只需让电工安装一次。
    2. **弱电计算核心 / 面板（Smart Core）**：通过安全的低压排针卡扣在底座上，包含无线 SoC（Wi-Fi/Thread/Zigbee）、传感器或按键面板。
  - **三大收益**：
    - **消除大众恐惧**：后续升级芯片（比如从 Gen1 升到 Gen2）或更换故障模块，用户只需像换电池一样拔下弱电模块，完全不接触 220V 强电。
    - **大幅削减 SKU 与认证成本**：昂贵的强电安规认证只需针对底座做一次；想要推出 Wi-Fi 版、Thread 版、带屏幕版或温湿度传感器版，只需更换低成本的弱电前盖。
    - **物理隔绝热源**：将发热的强电继电器留在暗盒深处，将对温度敏感的无线射频芯片和天线放在靠近外侧的弱电模块中，信号更好、散热更佳。
- **宽电压自适应（Universal Voltage）**：
  - 在电源设计上尽量实现 AC 100-240V 与 DC 12-48V 宽压兼容，减少因电压不同而派生的冗余型号。

四、 生态与商业冷启动：上市即“一等公民”

Shelly 的成功很大程度上归功于开源社区（尤其是 Home Assistant）的自来水推广。新品牌切忌在早期逼迫用户下载一个体验粗糙的自研 App。

1. **“Works with Home Assistant” Day-1 策略**：
   - 在硬件众筹或正式发售**之前**，先把产品的本地 API 文档开源，并由官方团队主动向 **Home Assistant、Zigbee2MQTT、ESPHome（提供官方硬件引脚配置文件 YAML）** 提交原生集成代码（PR）。
   - 只要极客发现在 HA 里你的设备能“秒发现、纯本地、零延迟”，并且官方居然主动提供 ESPHome 引脚定义图，你将在技术社区瞬间获得极高的信任度。
2. **用开放标准取代私有外设协议**：
   - 传感器广播直接采用 **BTHome v2** 标准（带加密的开源蓝牙广播协议），主控协议原生支持 **Matter**，工业侧原生支持 **Modbus TCP**。不造封闭轮子，让你的硬件成为任何生态都能直接吞下的“标准件”。
3. **商业变现闭环（C 端做口碑与规模，B 端做利润）**：
   - **C 端**：靠高性价比硬件 + 极致开放性走量，不收订阅费，建立品牌护城河。
   - **B 端**：针对公寓房东、酒店、中小型商业楼宇、光伏储能安装商，提供导轨式硬件 + **批量设备远程运维管理平台（Fleet Management）与能源管理（EMS）API**，通过商业级 SLA 服务和行业定制赚取高毛利。

五、 总结：新一代物联网硬件架构蓝图对照表

架构维度	Shelly 的做法（现状与痛点）	你的新产品架构方案（进阶破局）
无线与组网	主打 Wi-Fi（发热大/单火难），Gen4 的 Wi-Fi 与 Zigbee 互斥	**Wi-Fi 6 + Thread (IPv6) 双线并重**：强电大带宽用 Wi-Fi，暗盒/单火/低功耗用 **Thread**（兼具超低功耗与原生 IP 地址）
硬件与安全	早期 4MB Flash 捉襟见肘；普通继电器持续耗电发热	**8MB~16MB Flash 起步**；标配**过零检测 + 磁保持继电器**，大幅降低暗盒温升
工业结构	一体化强电盒（普通人不敢装，换代需重接强电，SKU 极多）	**强弱电分离插拔架构**：通用强电底座（一次安装/一次安规）+ 可热插拔弱电智能核心
通信协议栈	纯文本 JSON-RPC（对人友好，但占内存、默认无加密）	**双轨制 RPC**：对 Web/HTTP 暴露 **JSON-RPC**；对中枢高频同步采用 **CBOR/Protobuf 二进制 + Noise 端到端加密**
边缘计算	mJS 脚本引擎（门槛高，写错易引发内存不足）	**可视化 JSON 本地规则状态机**（覆盖 90% 需求）+ **带硬件看门狗防变砖沙盒**的轻量脚本
冷启动生态	逐步演进支持开源平台	**首发即开源**：上市前完成 Home Assistant 原生集成、BTHome 支持，并公开官方 ESPHome 引脚模板

User prompt: 在Shelly Group 的这个市场中，JSON-RPC 与 RESTful 相比，哪个好?

Response: 在 Shelly Group 所处的**智能家居后装模组与物联网边缘硬件市场**中，答案非常明确：**以 JSON-RPC 作为底层核心架构，远优于 RESTful**。

事实上，Shelly 自身的发展史就是这一结论最有力的证明——**Shelly 在第 1 代（Gen1）产品中采用的就是传统的 RESTful 架构，但在第 2 代（Gen2）及后续产品中，不惜打破向下兼容也要全面重构为 JSON-RPC 2.0**。

为什么在互联网 Web 开发中大行其道的 RESTful，到了 Shelly 这样的物联网硬件市场却败给了 JSON-RPC？我们可以从该市场的 **5 个核心行业特征** 来做深度对比：

一、 为什么在这个市场中 JSON-RPC 完胜 RESTful？（4 大核心维度）

1. 跨传输通道复用能力（决定研发与维护成本）

- **RESTful 的致命伤：与 HTTP 强绑定**。 RESTful 严重依赖 HTTP 协议的特性（如 URL 路径、GET/POST/PUT/DELETE 动词、HTTP 状态码 200/404/500、Header 头）。但在 Shelly 的市场里，用户绝不仅仅使用 HTTP，还会大量使用 **MQTT**（跨网段/云端）、**WebSocket**（Home Assistant 实时长连接）、**蓝牙 BLE GATT**（配网与近场控制）以及 **UART 串口**（工业对接）。
    - *后果*：在 Gen1 时代，Shelly 必须为 HTTP 写一套 REST 接口，再为 MQTT 单独设计一套完全不同的 Topic 规范。固件开发者要维护两套解析逻辑，第三方开发者换个通信方式就得重写代码。
- **JSON-RPC 的优势：协议无关性（Transport-Agnostic）**。 JSON-RPC 把所有的调用信息（调用谁 `method`、传什么参 `params`、请求编号 `id`）全部打包在一个自包含的 JSON 字符串里。这个 JSON 包就像一个标准集装箱，无论底层的运输工具是 HTTP、WebSocket、MQTT 还是串口线，**载荷格式 100% 相同**。

2. 双向实时通信与异步并发（决定智能家居响应体验）

- **RESTful 的局限：单向同步“请求-响应”**。 智能家居是典型的**事件驱动（Event-Driven）场景**——**比如有人按了一下墙上的物理开关，或者电热水器突然功率过载，设备必须主动、毫秒级**地通知中枢（如 Home Assistant）。RESTful 基于短连接，无法原生实现服务端主动推送；而在长连接通道上，RESTful 缺乏对乱序异步响应的匹配机制。
- **JSON-RPC 的优势：原生支持“异步 ID 匹配”与“无 ID 通知（Notification）”**。
    - 在一条复用的 WebSocket 或 MQTT 通道中，客户端可以一口气发出 10 个不同 `id` 的 JSON-RPC 请求，设备处理完哪个就先返回哪个（通过 `id` 对号入座），不会阻塞队列。
    - 当物理开关被按下时，设备只需主动发出一条不带 `id` 的 JSON-RPC `NotifyStatus` 报文，天然契合全双工实时推送。

3. 硬件控制语义的契合度（“动作导向” vs “资源导向”）

- **RESTful 是“资源导向（面向名词 CRUD）”**： RESTful 非常适合互联网数据库的增删改查（比如获取用户信息 `GET /users/1`）。但智能硬件绝大多数操作是“瞬态动作（动词）”——比如 `Reboot`（重启）、`Toggle`（翻转开关）、`Calibrate`（校准卷帘门行程）、`ResetEnergyCounters`

（清零电量统计），或者“打开开关并在 5.5 秒后自动关闭”（toggle_after）。如果强行套用 RESTful 规范，往往会设计出非常别扭的 URL 和状态映射。

- **JSON-RPC 是“过程/动作导向（面向动词 Method）”：** 硬件能力被直接抽象为 <组件>.<方法>（如 Switch.Toggle、Cover.Calibrate、Sys.Reboot）。这种设计直白、直观，完全符合硬件工程师和自动化脚本编写者的直觉。

4. 嵌入式芯片（MCU）的资源开销

- 在成本敏感的智能硬件中（几十元人民币的售价，运行在几兆内存的 ESP32 单片机上），如果采用纯正的 RESTful，固件里的 Web Server 需要包含复杂的 URL 路由树解析器、正则匹配、HTTP 动词分发器，同时为了支持 MQTT 还得再塞进一套解析逻辑。
- 采用 JSON-RPC 后，单片机内部只需要**一个统一的 JSON 解析器 + 一张扁平的函数指针映射表（Dispatcher）**。不管是来自网页、MQTT 还是本地脚本的指令，全部汇入同一个 C 语言入口函数分发，极大精简了固件代码体积（Flash 占用）并降低了内存碎片化风险。

二、 RESTful 唯一胜出的场景在哪里？

尽管 JSON-RPC 在架构上全面占优，但在 Shelly 的客户群中，有一个场景 **RESTful（准确地说是类 REST 的 HTTP GET）依然不可替代**：

- **极简客户端与“无代码”草根生态（Webhook / 快捷指令）** 许多初级玩家或老旧设备（如早期的网络摄像机报警触发、苹果 iOS 快捷指令、简单的路由器脚本、浏览器书签）只会发起最简单的 **HTTP GET URL 请求**，它们要么不支持构造复杂的 HTTP POST JSON Body，要么配置起来极其繁琐。
  - 用 RESTful 风格：只需要填一行网址 http://192.168.1.50/relay/0?turn=on 就能开灯。
  - 用纯正 JSON-RPC：必须配置 POST 请求、设置 Content-Type: application/json，还要手写带有双引号转义的 JSON 字符串，对非程序员极不友好。

三、 全景对比总结表（针对智能硬件/IoT 市场）

评估维度	RESTful 架构（如 Shelly Gen1）	JSON-RPC 2.0 架构（如 Shelly Gen2/3/4）	胜出者与原因
多协议复用（HTTP/WS/MQTT/串口）	❌ 差（与 HTTP 强耦合，其他协议需另起炉灶）	✅ **极佳**（一套报文走遍所有有线/无线通道）	**JSON-RPC**（省去多套接口维护成本）
实时状态推送（Local Push）	❌ 不支持（需靠轮询或外挂其他推送协议）	✅ **原生支持**（通过 WebSocket/MQTT 发送 Notification）	**JSON-RPC**（满足智能家居零延迟需求）
硬件控制语义表达	⚠️ 一般（CRUD 名词模型表达硬件动作较别扭）	✅ **极佳**（组件.动作 天然契合硬件控制与校准）	**JSON-RPC**（接口语义清晰直观）
单片机（MCU）固件开销	⚠️ 偏高（需解析复杂 URL 路由树 + 多协议逻辑）	✅ **极低**（单一入口 /rpc，单一 JSON 分发器）	**JSON-RPC**（节省宝贵的 Flash 与 RAM）
自省能力（自动发现接口）	❌ 弱（需依赖外部笨重的 OpenAPI/Swagger 文档）	✅ **强**（调用 Shelly.ListMethods 即可返回支持列表）	**JSON-RPC**（方便第三方插件自动适配）
浏览器地址栏 / 简易 Webhook 调用	✅ **极佳**（一条 URL 直接触发，无需拼装 Body）	❌ 繁琐（标准规范要求构造 JSON 请求体）	**RESTful**（对低级客户端与普通用户更友好）

四、 最终结论与行业最佳实践

在 Shelly Group 所在的智能硬件市场中，**没有必要在两者之间做非黑即白的二选一，最优解是 Shelly 目前采用的“内核 JSON-RPC + 表层 REST 语法糖”混合模式**：

1. **95% 的架构底座坚定选择 JSON-RPC：** 将设备内部所有的状态读取、配置修改、动作执行、事件推送全部统一为标准的 JSON-RPC 规范，跑在 WebSocket、MQTT、蓝牙、串口和 HTTP POST /rpc 之上。这是保证系统高内聚、低耦合、跨协议一致性的基石。
2. **5% 的边缘入口保留 HTTP GET 映射（吸收 RESTful 的优点）：** 在设备的 HTTP 服务器上做一个简单的转换层——当收到形如 GET /rpc/Switch.Set?id=0&on=true 的 URL 请求时，自动把 URL 路径里的 Switch.Set 和查询参数转化为内部的 JSON-RPC 调用。

这样既在底层享受了 JSON-RPC 跨协议统一与全双工通信的架构红利，又在表面保留了 RESTful 一行 URL 即可调用的极简体验。

User prompt: 列举Shelly Group的目标市场，特别是主要的、较新的、高端的市场，分析这些市场的特征。减少SKU, 重组Shelly Group当前所有的产品或重新设计产品，分析需要哪些产品即可满足这些市场。详细列出这些产品的规格。

Response: 要解决 Shelly Group 目前因快速迭代而产生的“SKU 极度膨胀（在售超 100 款型号）”问题，同时精准覆盖从大众 DIY 到高端前装的各个市场，必须从**顶层市场需求拆解**出发，通过**硬件平台归一化、芯片多模化、结构模块化**来重构产品矩阵。

以下为您详细拆解 Shelly Group 的目标市场特征，并给出将 100+ 个混乱 SKU 压缩至 **4 大产品线、12 款核心标准品** 的重组方案及详细硬件规格书。

第一部分：Shelly Group 目标市场全景与深度特征分析

Shelly Group 的业务已从早期的“极客玩具”演进为覆盖 C 端、B 端安装商及商业能源的立体矩阵。其目标市场可划分为 **主要市场（基本盘）**、**较新市场（高增长引擎）** 和 **高端市场（高毛利破局点）** 三大层级：

1. 主要市场（核心基本盘：贡献核心现金流与出货量）

- **① C 端 DIY 后装改造与开源极客市场（DIY & Prosumer Retrofit）**
  - **区域分布**：以欧洲（德奥瑞 DACH 地区、意大利、北欧）和北美为主。
  - **市场特征**：
    - **存量老房多**：不愿破坏原有墙面装修和物理开关面板，强依赖能够塞进狭小暗盒（60mm 欧标盒）的微型模组，且老房常面临无零线（单火线）痛点。
    - **极度注重隐私与本地化**：受欧洲 GDPR 及技术文化影响，排斥强制上云，深度绑定 Home Assistant、Node-RED 等开源中枢。
    - **价格敏感但懂技术**：拒绝为高溢价品牌网关买单，追求单品高性价比与极致的可编程性（API/脚本）。
- **② B 端专业电工与住宅安装商市场（Professional Installers & Electricians）**
  - **市场特征**：
    - **配电箱集中作业**：电工更倾向于在配电箱（DIN 导轨）内集中施工，而非逐个撬开客户家的墙壁开关。
    - **零返工率与安装效率**：在欧美，电工上门人工费极高（100~150 欧元/小时）。他们不在乎模块贵 10 欧元，但极度在乎**免工具快接端子（Push-in）、硬件稳定性、过温/过载保护**以及严苛的安规认证（VDE/UL/CE）。
    - **有线网络刚需**：金属配电箱会严重屏蔽 Wi-Fi 信号，因此对 **RJ45 以太网口** 有强烈刚需。

2. 较新市场（核心增长引擎：乘欧洲能源转型与轻商改造东风）

- **③ 家庭能源管理与微电网协同市场（HEMS、阳台光伏、热泵与 EV 充电）**
  - **背景**：欧洲能源危机后，德国等地普及“阳台光伏（Balkonkraftwerke）”、动态分时电价（如 Tibber/Nord Pool）以及热泵补贴政策。
  - **市场特征**：
    - **双向高精度计量**：需要实时监测电网买入与光伏卖出的双向功率（四象限计量），精度要求达到 1% 甚至 0.5S 级。
    - **大功率负载均衡与本地闭环**：需要在电价为负或光伏富余时，通过本地脚本自动触发大功率接触器，联动热水器、SG-Ready 热泵或电动汽车充电桩（EVSE），并在家庭总功率接近空气开关阈值时自动卸载负荷（Load Shedding）。
- **④ 轻商业、中小型楼宇与多户公寓/短租管理（Light Commercial, SMB & MDU）**
  - **场景**：小型办公室、连锁商铺、中小型酒店、长租公寓及 Airbnb 房东。
  - **市场特征**：
    - **买不起传统 BMS，看不上纯家用产品**：传统西门子/施耐德楼宇自控（BMS）造价动辄数万欧元；中小型业主需要低成本的导轨/暗盒方案，但要求原生支持 **Modbus TCP/RTU、BACnet 或 MQTT** 集中监管。
    - **防篡改与批量运维（Fleet Management）**：需要限制租客权限、远程监测空调/取暖器能耗异常，并具备断网自治能力。

3. 高端市场（高毛利破局点：切入前装豪宅与定制集成）

- **⑤ 高端住宅前装与 KNX 混合生态市场（Luxury Pre-wired & KNX Hybrid）**
  - **市场特征**：
    - **确定性与有线总线信仰**：高端别墅与商业建筑由系统集成商（SI）主导，底层坚持使用 **KNX 双绞线/IP** 或 **DALI 数字照明总线**。

- **传统系统的“智能化升级”痛点**：传统 KNX 执行器价格高昂且难以直接接入现代无线生态（如 Apple Home、Matter 或蓝牙传感器）。市场急需一种**既能用标准 ETS 软件编程接入 KNX 总线，又能同时兼任 Matter/IP/蓝牙桥梁**的高性价比混合执行器。

- **⑥ 专业全宅定制安装与设计师照明市场（Custom Integration & Architectural Lighting）**
  - **市场特征**：对灯光调光曲线的平滑度（0.1% 深度无频闪调光）、多色温（Tunable White / DALI DT8）、窗帘静音同步控制以及墙面中控屏的工业设计美学有极高要求，常需对接 Control4、Crestron（快思聪）或 Savant 系统。

第二部分：大刀阔斧削减 SKU —— 产品重组核心逻辑

1. 诊断：Shelly 为什么会有 100+ 个 SKU?

目前 Shelly 的产品线存在严重的“维度正交爆炸”：

- **代际同堂**：Gen1、Plus (Gen2)、Gen3、Gen4 同时在售；
- **协议割裂**：同一款开关分成 Wi-Fi 版、Wave（Z-Wave）版、Pro KNX 版；
- **尺寸冗余**：同一款 16A 继电器既卖标准尺寸，又卖 Mini 尺寸；
- **计量割裂**：每个型号都分成“不带电量计量（如 Shelly 1)”和“带计量（如 Shelly 1PM)”，而一颗计量芯片在大规模量产下的 BOM 成本差异不足 0.2 美元。

2. 重组五大法则（将 100+ SKU 压缩至 12 个）

1. **芯片与射频大一统**：全线标配单颗多模旗舰 SoC（支持 **Wi-Fi 6 + BLE 5.4 + 802.15.4 Thread/Zigbee 并发**，内建 **16MB Flash + 8MB PSRAM**），通过双射频时分复用技术，彻底解决 Gen4 上“开启 Zigbee 就丢失 Wi-Fi/Web/IP”的顽疾。
2. **取消“标准版”与“Mini 版”区分**：所有单路/双路暗盒模块全面采用 **Mini 级微缩尺寸**，通过采用**磁保持继电器（Latching Relay）+ 硬件过零检测**解决小体积散热问题。
3. **取消“无计量版”，干湿节点明确分离**：所有湿节点（直接输出火线）产品**100% 标配功率计量（PM)**；仅保留一款纯干接点（Dry Contact）微模块用于车库门/壁挂炉。
4. **单火/零火合一**：调光器采用自适应取电架构，同一硬件同时兼容“有零线”和“无零线（单火线)”环境。
5. **导轨 Pro 系列整合为“全能母舰 + 扩展从机”**：不再生产 Pro 1/1PM/2/2PM/3 等十几个零散型号，统一为一款高集成度的 **4 路全能主控（自带以太网/PoE/Modbus/KNX IP）** 搭配低成本扩展模块。

第三部分：重构后的 12 款核心产品矩阵与详细规格书

通过以下 **4 大产品线、共 12 款标准 SKU**，即可 100% 覆盖上述所有目标市场：

```
重构后全景矩阵（共 12 个 SKU）
├──── A. 暗盒后装微模块系列 (In-Wall Core) ──── 4 SKUs（覆盖C端DIY、老房改造、高端照明）
├──── B. 导轨专业与前装系列 (DIN-Rail Pro) ──── 3 SKUs（覆盖电工配电、能源管理、轻商业、KNX豪宅）
├──── C. 免安装与交互中枢系列 (Plug & HMI) ──── 2 SKUs（覆盖大众消费者、全屋中控屏、温控面板）
└──── D. 超低功耗无线感知系列 (Ambient Sense) ── 3 SKUs（覆盖全屋传感、无线场景开关、暖气控温）
```

A 系列：暗盒后装微模块系列（In-Wall Core Series —— 4 SKUs）

通用底座规格（A1~A4 共享)：

- **主控芯片**：定制 RISC-V 多模 SoC（160MHz 双核，16MB Flash，8MB PSRAM)。
- **无线连接**：Wi-Fi 6 (802.11ax 2.4GHz) + Bluetooth 5.4 (支持 BTHome 网关) + IEEE 802.15.4 (原生支持 **Matter over Thread** 及 Zigbee 3.0 双栈并发)。
- **软件与接口**：内置 Gzip 单文件 Web UI；支持 JSON-RPC 2.0 与 CBOR 二进制加密双轨 API；内置带看门狗沙盒的本地自动化引擎。
- **安全保护**：内置 NTC 双点温控、硬件级过流/过压/过功率切断、UL94-V0 阻燃外壳、Push-in 免螺丝快接端子。

SKU 1：Shelly Core 1D（全能宽压干接点微模块）

- **替代原有型号**：Shelly 1, Plus 1, 1 Mini Gen3/Gen4, Shelly Uni, Wave 1。
- **目标市场**：DIY 车库门/门禁改造、燃气壁挂炉干接点控制、弱电自动化。
- **详细规格**：

- **供电输入**：宽电压自适应 AC 100–240V (50/60Hz) 或 DC 12–48V（免跳线帽自动识别）。
- **输出类型**：1 路无源干接点（Dry Contact / Potential-Free），触点与供电端实现 4000V 电气隔离。
- **负载能力**：最大 16A (240V AC) / 10A (30V DC)，内置硬件过零吸合（AC 模式下）与磁保持继电器（待机功耗 < 0.4W）。
- **外部接口**：1 路开关量输入（SW）+ 1 路微型数字扩展口（支持直连 DS18B20 温度探头或门磁传感器）。
- **尺寸**：**29 × 34 × 16 mm**（Mini 级超小体积，可轻松塞入任何拥挤暗盒）。

SKU 2：`Shelly Core 2PM`**（双路计量与窗帘二合一微模块）**

- **替代原有型号**：Shelly 1PM, Mini 1PM, Plus 2PM, 2PM Gen3/Gen4, Wave 2PM, Wave Shutter（**以双路紧凑体积直接向下兼容单路需求**）。
- **目标市场**：全屋灯光/插座控制、电动卷帘门/百叶窗控制、分路能耗监测。
- **详细规格**：
  - **供电输入**：AC 100–240V (50/60Hz) 或 DC 24–48V。
  - **输出与负载**：2 路湿节点输出；单路最大 12A，双路总计最大 **16A (3680W)**（支持在软件中将两路并联设置为“单路 16A 大功率模式”）。
  - **继电器技术**：双磁保持继电器 + 独立硬件过零检测（Zero-Crossing），彻底消除感性/容性负载打火与暗盒发热。
  - **电能计量**：双通道独立高精度功率、电压、电流、功率因数测量（精度 ±1%）。
  - **工作模式**：软件一键切换 **“双路开关模式”** 或 **“窗帘/百叶窗电机模式（含自动行程校准与叶片角度调节）”**。
  - **尺寸**：**37 × 42 × 16 mm**。

SKU 3：`Shelly Core Dimmer`**（单零火自适应智能调光微模块）**

- **替代原有型号**：Shelly Dimmer 2, Plus Wall Dimmer, Gen3 Dimmer, Shelly 1L / 2L（单火线系列）。
- **目标市场**：欧洲/北美无零线老房照明改造、高端商业/住宅无频闪调光。
- **详细规格**：
  - **接线兼容性**：**零火线（With Neutral）与单火线（No-Neutral）自适应**，最低支持 3W 负载免旁路电容（Bypass）稳定运行。
  - **调光技术**：双 MOSFET 架构，支持 **后沿切相（Trailing Edge）与前沿切相（Leading Edge）** 自动识别与切换，支持 0.1%–100% 深度平滑调光与软启动。
  - **负载能力**：100–240V AC，支持 200W LED 负载或 300W 卤素灯/白炽灯，内置功率计量（PM）。
  - **输入接口**：支持双路复位按键（Push Button）或传统翘板开关调光。
  - **尺寸**：**37 × 42 × 16 mm**。

SKU 4：`Shelly Core LightBus`**（多协议专业照明控制器：DALI / 0-10V / RGBW）**

- **替代原有型号**：Shelly RGBW2, Plus RGBW PM, Plus 0-10V Dimmer, DALI Dimmer Gen3。
- **目标市场**：高端住宅设计师照明（线性灯带、色温灯）、商业办公室 DALI / 0-10V 灯盘与新风控制。
- **详细规格**：
  - **供电输入**：DC 12–48V 宽压输入（针对 DALI/0-10V 模式可通过外接微型 AC/DC 适配器取电）。
  - **三合一软件可配置输出接口**：
    1. **PWM 灯带模式**：4 路大功率 MOSFET 输出（单路 5A，总计 16A），支持 RGBW、RGB+CCT、双色温（Tunable White）或 4 路独立单色灯带，内置总功率计量。
    2. **0-10V / 1-10V 模拟调光模式**：提供 2 路独立主动/被动 0-10V 信号输出（控制商用驱动器或风阀）。
    3. **DALI-2 主控模式**：内置 DALI 总线电源（可驱动最多 16 个 DALI-2 镇流器，支持 DT6 调光与 DT8 色温控制）。
  - **尺寸**：**37 × 42 × 16 mm**。

B 系列：导轨专业与高端前装系列（DIN-Rail Pro Series —— 3 SKUs）

通用底座规格（B1~B3 共享）：

- **工业级通信大满贯**：除包含 A 系列全部无线能力（Wi-Fi 6 / Thread 边界路由器 / BLE 网关）外，全系标配 **100M RJ45 以太网口（支持 IEEE 802.3af PoE 受电）** + **RS485 硬件接口（Modbus RTU）**。
- **高端与商业协议**：原生支持 **Modbus TCP**、**KNX IP（支持导入 ETS 项目文件）**、**BACnet/IP（轻量化子集）** 以及 **Matter Bridge**。
- **光学级联总线（Side-Bus）**：模块侧面带有镀金触点，多模块并排安装在 DIN 导轨上时，可通过卡扣共用低压电源与高速通信总线，免去跳线烦恼。

SKU 5：`Shelly Pro 4PM Ultra`**（四路全能导轨主控与窗帘母舰）**

- **替代原有型号**：Pro 1, Pro 1PM, Pro 2, Pro 2PM, Pro 3, Pro 4PM, Pro Dual Cover PM, Pro KNX 系列。
- **目标市场**：专业电工配电箱改造、高端别墅 KNX/Matter 混合前装、轻商业照明与空调群控。
- **详细规格**：
  - **安装宽度**：标准 **2-DIN 模数宽度（36mm）**（比现有 4PM 缩小一半体积）。
  - **电气与隔离**：4 路独立通道，**通道间支持不同相线（L1/L2/L3 三相混接）**！单路最大 16A，四路总计 **40A**，内置过零磁保持继电器。
  - **计量与显示**：4 路独立高精度电能计量；正面配备 **1.5 英寸高对比度电子墨水屏（E-ink）+ 物理操控按键**，断电也能显示回路标签名称，通电实时显示功率、IP 与二维码，方便电工现场调试与贴标。
  - **灵活角色**：每一对通道均可独立配置为“普通分路开关”或“带行程校准的卷帘门/窗帘控制器”。

SKU 6：`Shelly Pro 3EM Grid`**（三相四象限能源网关与微电网控制器）**

- **替代原有型号**：Shelly EM, 3EM, Pro EM-50, Pro 3EM, Pro 3EM-400, 3EM-63T。
- **目标市场**：家庭能源管理（HEMS）、阳台光伏防逆流、热泵（SG-Ready）与充电桩动态负载均衡、商业能耗审计。
- **详细规格**：
  - **安装宽度**：超紧凑 **1-DIN 模数宽度（18mm）**。
  - **计量能力**：三相四线制（兼容单相三回路独立测量），**四象限双向计量**（有功、无功、视在功率、功率因数、谐波畸变率 THD），测量精度达 **IEC 62053-22 Class 0.5S 级**，支持最高 50Hz/100ms 毫秒级功率刷新率（满足光伏逆变器快速防逆流要求）。
  - **模块化互感器接口（RJ12 统一插口）**：主机统一，按需选配卡扣式开口 CT 互感器（50A / 120A / 400A）或直插式铜排穿刺模块（63A），彻底消灭因电流量程不同而产生的多个主机 SKU。
  - **内置负载联动输出**：自带 **2 路 2A 无源干接点输出**，原生支持德国 **SG-Ready 热泵协议** 以及大功率交流接触器（Contactor）线圈驱动。

SKU 7：`Shelly Pro IO-8`**（导轨干接点与多传感工业扩展从机）**

- **替代原有型号**：Pro 3 (Dry Contact), Plus Add-on, Shelly I4 / Pro I4。
- **目标市场**：地暖分集水器阀门控制、多路安防干接点采集、机房/锅炉房多点温度监控。
- **详细规格**：
  - **安装与连接**：1-DIN 宽度（18mm）；可直接吸附在 `Pro 4PM Ultra` 或 `Pro 3EM Grid` 右侧通过侧边总线取电通信，也可通过顶部 DC 12-24V + RS485 独立运行。
  - **输出接口**：4 路 5A 独立无源干接点（用于控制地暖热电执行器、新风阀门或报警器）。
  - **万能输入接口**：8 路软件可配置输入端子，支持开关量（干接点/湿节点）、0-10V 模拟量传感器、4-20mA 工业变送器、以及最多 10 个并联 DS18B20 / PT1000 温度探头。

C 系列：免安装与人机交互系列（Plug & HMI Series —— 2 SKUs）

SKU 8：`Shelly Plug Ultra`**（微型全能计量插座与无线中继器）**

- **替代原有型号**：Shelly Plug, Plug S (Gen1/Gen2/Gen3), Plug US/UK/IT。
- **目标市场**：不敢动强电的纯大众消费者、租房群体、全屋无线信号盲区扩展。

- **详细规格**：
  - **负载与安全**：最大 **16A / 3680W**（美规 15A / 1800W），内置过零继电器、儿童安全门、插脚温度传感器（插头接触不良发热超过 85°C 自动断电报警）。
  - **光效交互**：360° 柔和导光环（可随实时功率由绿变红，或作为夜间人体感应小夜灯，亦可彻底关闭）。
  - **中枢扩展角色**：插上插座即自动成为全屋 **Matter over Thread 边界路由器（Border Router）+ BLE 蓝牙代理网关 + Wi-Fi 信号中继器**。

SKU 9：`Shelly Wall Hub`**（模块化智能中控屏与温控面板）**

- **替代原有型号**：Shelly Wall Display, Wall Display X2, Plus H&T（桌面/墙面温控）。
- **目标市场**：高端住宅墙面交互、中央空调/地暖温控器（Thermostat）、酒店客房控制面板。
- **详细规格**：
  - **强弱电分离插拔结构**：采用“通用暗盒强电底座（内置 2 路 5A 继电器，兼容欧标/国标/美标暗盒）+ 磁吸快拆屏幕面板”设计。
  - **屏幕与交互**：4 英寸 720×720 IPS 高清触控屏 + **顶部实体铝合金旋转编码器表冠（或底部两颗实体微动按键）**，解决纯触控屏盲操体验差的痛点。
  - **多维感知**：内置高精度温湿度传感器、环境光传感器、以及 **24GHz 毫米波微动雷达**（人走近自动亮屏，同时作为房间人体存在传感器输出给全屋系统）。
  - **系统能力**：内置独立本地中枢引擎，无需外部服务器即可直接绑定控制全屋 Shelly 设备、显示 `Pro 3EM Grid` 能源仪表盘，并具备完整的 PID 恒温器（地暖/风机盘管）控制逻辑。

D 系列：超低功耗无线感知与执行系列（Ambient Sense Series —— 3 SKUs）

通用底座规格（D1~D3 共享）：

- **双模低功耗通信**：同时支持 **BTHome v2（加密蓝牙广播，零延迟直连任何 Shelly 强电开关）** 与 **Matter over Thread**。

SKU 10：`Shelly Sense Multi`**（五合一模块化环境与安防传感器）**

- **替代原有型号**：BLU Door/Window, BLU Motion, BLU H&T, Shelly Flood（将 4 款传感器整合为 1 款）。
- **目标市场**：全屋自动化触发、安防监测、温湿度与漏水报警。
- **详细规格**：
  - **一机多用硬件架构**：单颗微型主机内部同时集成 **低功耗 PIR 人体移动传感器 + 光照度计 + 高精度瑞士 Sensirion 温湿度计 + 三轴加速度计（测窗户倾角/震动）+ 霍尔磁控开关（测门窗开合）**。
  - **形态切换配件**：包装内附带磁铁块（贴在门窗上即作为门窗+温湿度传感器）、万向转轴底座（立在墙角即作为人体移动+光照+温湿度传感器）、以及底部带水浸金属触点的底座（放在水槽下即作为漏水报警器）。
  - **续航能力**：采用单颗 CR2477 纽扣电池，典型续航 **5 年**。

SKU 11：`Shelly Sense Knob4`**（四键旋钮二合一无线场景开关）**

- **替代原有型号**：BLU Button1, BLU Tough, BLU RC Button 4, BLU Wall Switch。
- **目标市场**：床头/茶几免布线双控、灯光亮度/色温无极调节、窗帘开合控制。
- **详细规格**：
  - **交互设计**：标准开关面板内芯尺寸（可卡入主流品牌墙面边框，也可磁吸取下作为手持遥控器）；包含 **4 颗清脆微动按键 + 外圈磁编码无极旋钮**。
  - **断网直连（Direct Binding）**：支持通过蓝牙广播或 Thread Group 直接绑定 `Core 2PM` 或 `Core Dimmer`——**即使家里路由器断电、中枢死机，依然可以毫秒级按键开灯、旋转调光**。
  - **防护等级**：IP54 防尘防水，CR2032 电池续航 3 年以上。

SKU 12：`Shelly Climate TRV`**（超静音智能暖气片温控阀）**

- **替代原有型号**：Shelly TRV, BLU TRV。

- **目标市场**：欧洲存量住宅分室采暖节能改造、公寓集中供暖管控。
- **详细规格**：
  - **连接能力**：无需专用网关（告别 BLU TRV 必须捆绑专用 USB 网关的弊端），直接通过 Thread 或家里任意一台强电 Shelly 开关的蓝牙网关接入系统。
  - **精准控温**：原生支持与房间内的 `Shelly Sense Multi` 或 `Wall Hub` 直接配对，以房间中心的真实温度代替暖气片旁边的虚高温度进行闭环 PID 调节。
  - **电源设计**：可拆卸锂电池包（Type-C 充电，充一次电可用 2 个采暖季）或兼容 2 节标准 AA 电池。

第四部分：总结——12 款核心产品对目标市场的覆盖映射表

通过这套重组方案，Shelly Group 的产品数量缩减了 **85% 以上**（极大降低了供应链备货成本、安规认证成本与消费者选购认知负担），但对五大目标市场的覆盖深度反而大幅增强：

重构后的核心 SKU	C端 DIY 与老房改造	B端专业电工配电	家庭能源 (光伏/热泵)	轻商业与公寓管理	高端前装 (KNX/定制照明)
1. Core 1D (干接点)	✅ 车库门/门禁/壁挂炉	✅ 弱电/接触器触发	✅ 简易热泵启停	✅ 门禁电磁锁控制	⚪ 辅助干接点接入
2. Core 2PM (双路计量/窗帘)	✅ **主力走量爆款**	✅ 暗盒分路改造	✅ 单一电器功耗监测	✅ 客房灯光/窗帘	✅ Matter/Thread 窗帘
3. Core Dimmer (单零火调光)	✅ **解决老房无零线痛点**	✅ 传统灯具升级	⚪	✅ 餐厅/商铺调光	✅ 切相深度平滑调光
4. Core LightBus (DALI/0-10V/RGBW)	✅ 氛围灯带 DIY	✅ 商业灯盘/风阀	⚪	✅ 办公楼 DALI 照明	✅ **高端设计师照明核心**
5. Pro 4PM Ultra (导轨四路母舰)	⚪ 高阶玩家配电箱	✅ **电工装机首选**	✅ 大功率分路卸载	✅ **轻商配电柜主控**	✅ **原生 KNX + Matter 桥梁**
6. Pro 3EM Grid (三相双向能源表)	✅ 全屋总电表监测	✅ 配电箱标准配置	✅ **光伏/热泵/充电桩核心**	✅ 商铺/租户分表计费	✅ 豪宅能源管理中枢
7. Pro IO-8 (导轨I/O扩展)	⚪	✅ 地暖分集水器控制	✅ 蓄热水箱多点测温	✅ 楼宇消防/水泵监测	✅ 别墅暖通阀门集中控制
8. Plug Ultra (智能计量插座)	✅ **零门槛小白爆款**	⚪	✅ 家电级能耗追踪	✅ 办公设备定时断电	✅ 隐蔽处 Thread 信号中继
9. Wall Hub (模块中控温控屏)	✅ 替换传统墙面温控器	✅ 标配房间温控面板	✅ 家庭能源可视化看板	✅ 酒店客房温控面板	✅ **高颜值墙面交互终端**
10. Sense Multi (五合一传感器)	✅ 全屋自动化触发	⚪	⚪	✅ 无人关空调/安防	✅ 隐形环境感知节点
11. Sense Knob4 (按键旋钮开关)	✅ 免布线双控/床头开关	✅ 省去开槽布线人工	⚪	✅ 灵活改造办公开关	✅ 桌面/墙面调光旋钮
12. Climate TRV (暖气片温控阀)	✅ 欧洲冬季采暖刚需	✅ 采暖系统升级	✅ 配合热泵分室控温	✅ 公寓房东控温节能	⚪

User prompt: 简化后的SKU，哪些没有Wi-Fi，只有Thread（IPv6）的产品，是如何解决WebUI的问题的，还是不需要解决这个问题了?

Response: 在重构后的 12 款核心 SKU 矩阵中，并不是所有设备都塞入了高功耗的 Wi-Fi。针对“哪些产品没有 Wi-Fi、只有 Thread（及蓝牙 BLE）”，**以及**“没有 Wi-Fi 的情况下如何解决 Shelly 最具代表性的内置 WebUI 问题”，这恰恰是新一代物联网架构中最核心的工程设计权衡。

简而言之：**对于日常 90% 的基础功能，确实不再需要依赖内置 WebUI；但对于剩下的 10% 极客高阶配置（校准、修改底层阈值、OTA、调试），我们不能放弃 WebUI 体验，而是通过“IPv6 极简缓存”、“影子代理（Shadow UI）”和“WebBLE 浏览器蓝牙直连”三大技术手段，在无 Wi-Fi 环境下完美重构了 WebUI。**

以下为您详细拆解具体的 SKU 归属与底层技术实现方案：

一、 简化后的 12 款 SKU 中，哪些没有 Wi-Fi（或默认关闭 Wi-Fi）？

在 12 款产品中，**纯正只用 Thread（IPv6）+ BLE（蓝牙）而不使用 Wi-Fi 的产品**分为两类（共涉及 **4 款 SKU**）：

1. 硬件级“彻底无 Wi-Fi”：D 系列超低功耗产品（3 款 SKU）

- **包含型号**：
  - **SKU 10：**`Shelly Sense Multi`（五合一环境与安防传感器，CR2477 纽扣电池供电）
  - **SKU 11：**`Shelly Sense Knob4`（四键旋钮无线场景开关，CR2032 纽扣电池供电）
  - **SKU 12：**`Shelly Climate TRV`（智能暖气片温控阀，AA/锂电池供电）
- **为什么砍掉 Wi-Fi?**
  - Wi-Fi 射频在握手和发包时的瞬时峰值电流高达 **150mA～300mA**，而一颗 CR2032/CR2477 纽扣电池的最大持续放电电流仅为几毫安（强行拉起 Wi-Fi 会直接导致电池电压骤降触发芯片断电重启）。
  - 因此，这 3 款产品在硬件上直接选用超低功耗的 **IEEE 802.15.4 (Thread) + BLE 5.4 双模芯片**（如 Silicon Labs MG24、Nordic nRF54 或乐鑫 ESP32-H2），峰值电流控制在 10mA 以内，休眠电流低于 2 微安（μA）。

2. 场景级“自动关闭 Wi-Fi”：处于“单火线（无零线）”模式下的调光器（1 款 SKU）

- **包含型号**：
  - **SKU 3：**`Shelly Core Dimmer`（单零火自适应调光微模块，在**无零线 No-Neutral** 接线环境下）。
  - *(注：A 系列的其它型号如果被极客用户手动设置为“纯 Thread 低功耗/低发热模式”，也属于此类)*。
- **为什么关闭 Wi-Fi?**
  - 在没有零线的老房子里，模块必须串接在灯具回路中“偷电”维持生命。如果电流超过几毫安，LED 灯具在关闭状态下就会出现讨厌的“鬼火闪烁（Ghosting）”。
  - 因此，当 `Core Dimmer` 上电自检发现处于**单火线模式**时，固件会**自动熔断（关闭）高功耗的 Wi-Fi 射频**，仅开启毫瓦级的 **Thread（IPv6）+ BLE 射频**，从而实现免旁路电容（No Bypass）的极致稳定运行。

二、 痛点剖析：在只有 Thread（无 Wi-Fi）的设备上做 WebUI，难在哪里?

为什么传统厂商一旦不用 Wi-Fi（比如改用 Zigbee 或蓝牙），就会直接砍掉设备的内置网页后台（WebUI）？ 如果我们在 Thread 设备上强行照搬 Wi-Fi 时代的 WebUI，会遇到两个物理定律级别的障碍：

1. **物理带宽瓶颈（250 kbps vs 150 Mbps）**： Wi-Fi 的物理速率高达上百兆，传一个 1MB 的网页只需几十毫秒；而 Thread 底层采用的 IEEE 802.15.4 物理层总带宽只有 **250 kbps**，扣除 6LoWPAN 分片、IPv6 头和 TCP 握手开销后，**实际可用吞吐量只有约 15～25 KB/s**。如果网页文件有几百 KB，用户打开网页会卡顿十几秒甚至超时断开。
2. **电池休眠断联（Sleepy End Device 难题）**： D 系列的电池设备属于 **SED（休眠终端设备）**，为了维持 3～5 年续航，它在 99.9% 的时间里射频是彻底断电关闭的。此时如果用户在电脑浏览器里输入它的 IP 地址，请求根本无法到达设备，因为设备正在“睡觉”。

三、 破局方案：不需要解决这个问题了吗？还是有更聪明的解法?

答案是：**分层次解决——将“日常高频交互”与“低频工程配置”解耦，并用三套现代前端与网络技术化解上述两大物理瓶颈。**

1. 为什么说 90% 的场景下“确实不再需要打开 WebUI 了”？

在 Shelly Gen1/Gen2 时代，用户之所以频繁打开电池传感器（如老款 Shelly H&T、Door/Window）的内置 WebUI，是因为当时的 Wi-Fi 私有协议如果不配好 MQTT 或 Webhook，智能家居平台就拿不到数据，而且每次进 WebUI 都得先去用手按一下传感器唤醒 Wi-Fi，体验极其糟糕。

而在新架构下，这 4 款产品全面支持 **Matter over Thread** 与 **BTHome v2**：

- **即插即用**：通过 Matter 或 BTHome 接入 Home Assistant、Apple Home 或 Shelly 自家中枢后，温度、湿度、电量、按键事件、TRV 目标温度等所有标准属性，都通过标准集群（Clusters）原生呈现在平台界面上。
- **结论**：对于普通消费者日常看数据、设自动化，**彻底不需要再进设备的单机 WebUI**。

2. 但针对极客最爱的 10% 高阶配置，我们用“三级自适应 WebUI 架构”完美保留！

极客和专业安装商依然需要修改底层参数（例如：修改传感器的温湿度校准偏移量、调整 PIR 盲区时间、配置跨设备直接绑定 Direct Binding、离线上传固件包）。我们针对不同供电形态采用以下 3 种解法：

解法 A：针对常供电/单火线节点（如 Core Dimmer 单火模式）——【原生 IPv6 直连 + 极简 Gzip 缓存 WebUI】

Core Dimmer 虽然在单火线下关掉了 Wi-Fi，但它有市电维持，在 Thread 网络中属于 **常在线节点（Router / REED）**，随时可以响应网络请求。

- **跨网段无缝访问**：Thread 是原生 IPv6 网络。只要家里有任何一台 **Thread 边界路由器（TBR）**（比如 Apple HomePod、Google Nest、Home Assistant 网关，或者同系列的 Shelly Plug Ultra / Pro 4PM Ultra），边界路由器就会自动把局域网 Wi-Fi/以太网与 Thread 网状网路由打通，并通过 mDNS/SRP 广播域名。
- **用户在电脑/手机浏览器直接输入**：http://shelly-dimmer-a1b2.local（或直接点击 IPv6 地址 http://[fdxx:xxxx::xxxx]），请求就会穿过边界路由器直达开关！
- **如何克服 250kbps 慢速瓶颈？**
  1. **微缩单文件打包**：WebUI 采用 Preact/Svelte 编写，编译压缩为单个 .html.gz 静态文件，体积严格控制在 **35KB～45KB**。
  2. **永久缓存策略（ServiceWorker + Immutable Cache）**：首次在浏览器打开时，通过 Thread 传输这 40KB 文件仅需 **约 1.5 秒**；一旦加载完成，浏览器的 ServiceWorker 会将该网页框架永久缓存在本地。
  3. **后续零延迟打开**：用户第二次打开该设备的 WebUI 时，网页直接从浏览器本地秒开（0 字节网络传输），前后端仅通过 WebSocket 交换几百字节的 **CBOR / JSON-RPC** 状态包，体验与 Wi-Fi 设备毫无二致！

解法 B：针对休眠电池节点（D 系列传感器/旋钮/TRV）——【局域网“影子数字孪生 WebUI”（Shadow Proxy UI）】

因为电池设备（Sense Multi / Sense Knob4 / Climate TRV）平时在睡觉，不能直接跑 HTTP Server。怎么让用户随时随地都能用浏览器改它的配置？

- **借船出海（宿主代理）**：只要用户家里有**任意一台**接市电的 Shelly 设备（如 Plug Ultra 插座、Core 2PM 开关、Pro 4PM 导轨或 Wall Hub 屏幕），这台强电设备就会自动充当所有电池设备的 **“影子宿主（Shadow Host）”**。
- **工作流（异步期望状态同步）**：
  1. 用户用浏览器打开家里那台强电 Shelly 的 WebUI，左侧菜单会直接列出其管辖下的所有低功耗设备（如 Sense Multi 客厅传感器）。
  2. 用户点击进入后，看到的是与独立设备完全相同的配置界面（由强电设备代为渲染）。
  3. 用户在网页上把“温度偏移量改为 -0.5°C”并点击保存——此时传感器虽然在睡觉，但强电设备会将这条 JSON-RPC 指令存入本地的 **待下发队列（Pending Queue，类似 AWS IoT Device Shadow 机制）**，网页立即提示“已保存，待设备唤醒时自动同步”。
  4. 当传感器在几秒或几分钟后按周期微唤醒（通过 **Thread CSL 协调采样监听** 或蓝牙轮询）时，强电设备在几毫秒内将新配置自动刷入传感器。用户完全无需跑到墙高处去按传感器的物理唤醒键！

解法 C：针对“只买了一个单品传感器”的纯极客——【基于 W3C WebBLE 的免 App 浏览器直连】

如果一个极客玩家**只买了 1 个** Shelly Sense Multi **传感器**（家里没有任何其它 Shelly 强电设备，也没有安装 Shelly 手机 App，只想用电脑配好参数后扔进 Home Assistant），怎么给他提供 WebUI？

- **利用现代浏览器的 Web Bluetooth API（WebBLE）**：
  - 现在的 Chrome、Edge（Windows / macOS / Linux / Android）浏览器原生支持通过 JavaScript 直接调用电脑或手机的蓝牙硬件与设备通信。
  - **操作体验**：用户在电脑浏览器打开一个离线可用的纯静态配置网页（例如 ble.shelly.local 或扫描说明书二维码打开一个纯本地运行的 PWA 页面），按一下传感器上的按钮唤醒蓝牙，网页上点击“连接设备”，浏览器就会直接通过蓝牙 GATT 连上传感器，渲染出完整的图形化 WebUI 并直接读写内部的 JSON-RPC 配置——**全程不沾云端、不装 App、不依赖 Wi-Fi**。

四、 全景汇总：无 Wi-Fi 产品的 WebUI 解决方案对照表

产品类型与对应 SKU	射频与供电状态	日常基础使用是否需要 WebUI?	高阶配置如何实现 WebUI 体验?
A/B/C 系列常规态 *(Core 1D/2PM, Pro, Plug 等)*	市电常供 **Wi-Fi 6 + Thread 双开**	按需使用	**传统直连模式**：直接通过 Wi-Fi IPv4 或以太网毫秒级加载内置 WebUI。
SKU 3: Core Dimmer	市电微电流取电	❌ **不需要**	**原生 IPv6 直连 + 浏览器缓存**：通过 Thread 边界路由器直接访问 http://xxx.local，加载 40KB 极简 Gzip 网页并由浏

产品类型与对应 SKU	射频与供电状态	日常基础使用是否需要 WebUI?	高阶配置如何实现 WebUI 体验?
(处于“单火线无零线”模式)	**关闭 Wi-Fi，仅开 Thread + BLE**	*(通过 Matter 直接接入平台控制)*	览器缓存，后续交互仅传百字节 RPC 帧。
SKU 7: `Pro IO-8` *(导轨扩展从机)*	导轨总线/DC供电 **侧边总线 / RS485 / Thread**	❌ **不需要**	**宿主统一渲染**：直接作为扩展选项卡，内嵌显示在左侧 `Pro 4PM Ultra` 主控母舰的 WebUI 中。
D 系列低功耗产品 *(SKU 10 传感器 / SKU 11 旋钮 / SKU 12 温控阀)*	电池供电（99.9% 休眠） **硬件无 Wi-Fi，仅 Thread + BLE**	❌ **不需要** *(通过 Matter / BTHome 原生暴露所有状态)*	**双保险方案**： 1. **有 Shelly 强电设备时**：采用 **“影子数字孪生 WebUI”**，在强电设备网页上随时修改配置，唤醒时自动下发； 2. **仅有单品时**：采用 **WebBLE 技术**，电脑/手机浏览器通过蓝牙直连设备渲染 WebUI。

User prompt: 在Shelly Group 的这个市场中，如果完全重构产品与服务，有比 JSON-RPC 更好的选择吗?

Response: 如果今天抛开所有历史包袱，从零完全重构 Shelly Group 的软硬件与云边服务体系，**确实有比纯文本 JSON-RPC 2.0 更好的选择**。

但这个“更好的选择”绝不是简单粗暴地换成纯二进制的 **Protobuf / gRPC**（那会彻底摧毁 Shelly 赖以成名的“免编译、一行 `curl` 命令即可调用”的极客与系统集成商生态），而是采用一套 **“以 CBOR（二进制 JSON）为原生内核 + 对齐 Matter 语义数据模型 + 边缘流式兼容 JSON + CoAP/Zenoh 无连接传输”** 的现代物联网同构架构。

以下从**纯 JSON-RPC 的四大架构天花板**、**四大现代替代方案横评**以及**终极重构蓝图**三个维度进行深度拆解：

一、 为什么完全重构时，纯 JSON-RPC 不再是最优解?

在 2021 年 Shelly 设计 Gen2 架构时，产品几乎全是 Wi-Fi 设备，JSON-RPC 是兼顾 Web 前后端与 MQTT 的最佳折中。但当我们把产品线扩展到 **Wi-Fi 6 + Thread (802.15.4) + BLE + 导轨总线** 的全场景矩阵时，纯文本 JSON-RPC 暴露出了 **4 个物理定律级别的瓶颈**：

1. **低功耗网状网（Thread / 802.15.4）的 MTU 分片灾难**
   - Wi-Fi 的以太网帧最大传输单元（MTU）高达 1500 字节，传几百字节的 JSON 毫无压力。
   - 但 Thread 底层的 IEEE 802.15.4 物理帧最大只有 **127 字节**（扣除帧头与安全开销后，单包净荷仅剩 **约 80 至 100 字节**）。一段仅包含 150 字节的简单 JSON-RPC 请求，在 Thread 网络中会被 6LoWPAN 强行拆分成 2 到 3 个数据分片发送——这不仅使丢包重传概率成倍增加、局域网延迟从 10ms 劣化到 80ms 以上，还会让纽扣电池设备的射频唤醒时间翻倍。
2. **MCU 算力内耗与二进制数据的“Base64 膨胀”**
   - 单片机（MCU）内部存储变量全部是二进制的（如 `uint8_t`、`float`）。使用 JSON-RPC 意味着设备每次通信都要在内存中分配动态字符串缓冲区，进行昂贵的浮点数转 ASCII 文本、引号转义与字符串哈希比对。
   - 更糟糕的是，JSON 不支持原生二进制字节流（Byte Array）。当通过 RPC 传输蓝牙广播原始包、加密签名或 OTA 固件分片时，必须先编码为 **Base64 字符串**，体积白白膨胀 **33%**。
3. **TCP 连接数饥饿（Socket Exhaustion）与 TLS 内存爆炸**
   - Shelly 的 JSON-RPC 强依赖长连接 **WebSocket（基于 TCP）** 来实现本地实时状态推送。但一颗成本几美元的嵌入式芯片（如 ESP32 系列），其宝贵的 SRAM 通常只能同时维持 **8 到 12 个并发 TCP/TLS 连接**。一旦家里同时有 Home Assistant、Node-RED、中控屏 `Wall Hub`、多个手机 App 打开网页连接同一台开关，设备的 Socket 池就会瞬间耗尽并拒绝服务。
4. **缺乏原生的“订阅滤波（Deadband）”与行业标准语义**

- JSON-RPC 只有“过程调用（Call）”的概念，没有原生的属性订阅机制。Shelly 被迫打补丁发明了 `NotifyStatus`，但客户端无法向设备声明：“只有当功率变化超过 5W、且间隔大于 10 秒时才向我推送数据”。
- 此外，`Switch.Set` 依然是 Shelly 自创的私有方言，接入 Matter 或 HomeKit 时，固件内部还得再跑一层语义转换表。

二、 物联网领域四大现代替代方案深度横评

如果我们跳出 JSON-RPC，当前嵌入式与物联网领域有 4 种主流演进方向：

1. **Protobuf (nanopb) + 私有二进制帧**（代表作：ESPHome Native API）
   - **原理**：通过 `.proto` 文件预定义强类型数据结构，编译为极致紧凑的二进制流。
   - **致命伤**：**彻底丧失自描述性（Schema-Mandatory）**。如果外部客户端没有提前拿到对应固件版本的 `.proto` 文件并编译代码，收到的一串二进制字节完全无法解析，普通用户再也无法用浏览器、手机快捷指令或简单 Webhook 控制设备。
2. **CBOR-RPC (IETF RFC 8949) + CDDL**（简明二进制对象表示）
   - **原理**：互联网工程任务组（IETF）专为受限物联网制定的“二进制版 JSON”国际标准（已被 Matter 底层凭证、WebAuthn 通行密钥、CoAP 广泛采用）。
   - **核心优势**：它与 JSON 的数据模型 **1:1 完全同构**（同样是键值对、数组、布尔、数字），但采用紧凑的二进制 TLV（类型-长度-值）编码：**无需预编译 Schema 即可直接解析**，原生支持 `byte[]` 二进制流和显式整数/浮点数，且在芯片上只需不到 **3KB 代码** 就能与标准 JSON 文本进行零损耗双向流式互转！
3. **Matter Interaction Model (TLV) + CoAP / OSCORE**（CSA 标准交互模型）
   - **原理**：抛弃自由命名的 RPC，将所有硬件功能标准化为 `Endpoint（端点）-> Cluster（功能簇）-> Attribute（属性）/ Command（命令）/ Event（事件）` 三级树，底层通过无连接的 **UDP（CoAP）** 传输，并用 **OSCORE / COSE** 对单个数据包做对象级加密（无需维持耗内存的 TLS 会话）。
4. **Eclipse Zenoh (Zenoh-Pico)**（新一代分布式数据网格协议）
   - **原理**：2025–2026 年在边缘计算与机器人领域异军突起的新一代协议（专为取代 MQTT + DDS + HTTP 查询而生）。
   - **核心优势**：协议头开销低至 **4 到 6 字节**，原生融合了 **发布/订阅（Pub/Sub）** 与 **远程查询/调用（Query/RPC）**。最强大的是它打破了 MQTT 的“中心代理悖论”——两台 Shelly 设备在局域网内无需任何 Broker 即可直接点对点通信；而当网络中存在 Zenoh 路由器（如中控屏或云端）时，又能自动跨网段路由。

评估维度	纯文本 JSON-RPC 2.0 (Shelly 现行)	Protobuf + Native API (ESPHome 模式)	Matter IM (TLV) + CoAP (UDP 标准模式)	CBOR-RPC + JSON 边缘转译 + CoAP/Zenoh (重构推荐)
单次开关控制报文大小	~95 Bytes (纯文本)	**~10 Bytes** (极小)	~18 Bytes (TLV 编码)	**~14 Bytes (字典压缩 CBOR)** / ~45 Bytes (标准 CBOR)
802.15.4 (Thread) 单包直达	❌ 易触发 6LoWPAN 分片	✅ 轻松单包直达	✅ 轻松单包直达	✅ **轻松单包直达（零分片）**
MCU 内存与解析开销	❌ 高（字符串解析 + Base64）	✅ 极低（静态结构体映射）	✅ 低（位级 TLV 解析）	✅ **极低（可原位 In-place 解析至栈内存）**
人类可读与 `curl` 简易调用	✅ **极佳（核心卖点）**	❌ **极差（必须编译 Schema）**	❌ 差（需专用 Matter 控制器）	✅ **极佳（HTTP 边缘 3KB 转换器自动映射为 JSON）**
并发控制端数量上限	⚠️ 低（受限于 TCP Socket 数量）	⚠️ 低（基于 TCP 长连接）	✅ **无上限（基于无连接 UDP）**	✅ **无上限（局域网走 UDP / CoAP / Zenoh）**
无中心网关点对点联动	⚠️ 仅支持粗糙的 HTTP Webhook	❌ 强依赖中枢服务器	✅ 原生支持 Group 组播绑定	✅ **原生支持 UDP 组播与无代理 P2P 订阅**

三、 终极重构方案：如何设计一套全面超越 JSON-RPC 的新架构？

如果由我们来主导 Shelly 全系产品与服务的重构，最佳架构不是彻底抛弃 JSON-RPC 的开发体验，而是将其升级为 **“内核二进制化（CBOR）、语义标准化（Matter Clusters）、传输无连接化（CoAP/Zenoh）”** 的三层融合架构：

第一层：序列化与接口层 —— 【原生 CBOR-RPC 内核 + HTTP 边缘 JSON 自动转译】

我们将固件内部所有的通信、存储、脚本调用全部重构为基于 **CBOR（RFC 8949）** 的二进制结构：

1. **对机器与低功耗网络（Thread / BLE / 局域网中枢 / 云端）—— 100% 走纯二进制 CBOR**：
   - 在 Thread、蓝牙 GATT、MQTT 或面向 Home Assistant 的高速通道上，设备直接收发二进制 CBOR 报文。
   - **启用 CBOR 静态字典整数映射（Packed CBOR）**：将高频使用的 JSON 键名（如 `"method"`、`1`，`"params"`、`2`，`"Switch.Set"`、`101`）直接映射为 1 字节的整数标签。原本 95 字节的 JSON-RPC 请求瞬间缩小到 **12 至 15 字节**，CPU 解析耗时降低 **80% 以上**，且可直接塞进单个蓝牙广播包或 Thread 帧中。
2. **对人类开发者与第三方简易集成（HTTP / WebUI）—— 网关层 3KB 流式转译为 JSON**：
   - 因为 CBOR 与 JSON 是 1:1 同构的，我们在强电设备的 HTTP Server 入口处放置一个仅占 **3KB Flash 的流式转译器（Stream Transcoder）**。
   - 当开发者用 `curl`、浏览器或 Webhook 发送 `Content-Type: application/json`（甚至 `GET /rpc/Switch.Set?id=0&on=true`）时，HTTP 入口在读取网络字节流的同时，实时将其转译为内部 CBOR 令牌交给内核处理；返回时再将 CBOR 自动格式化为可读的 JSON 文本。
   - **效果**：极客和集成商依然以为自己在调世界上最好用的 JSON-RPC，但整个系统的底层已经变成了极致高效的二进制引擎！

第二层：数据模型与语义层 —— 【用“属性树 + 增量订阅”取代孤立的“过程调用”】

抛弃原本零散的 `<Component>.<Method>` 命名，在固件内部直接采用与 **Matter Interaction Model（交互模型）同源的四元原语**：

- `Read`**（读属性）** / `Write`**（写属性）** / `Invoke`**（执行动作）** / `Subscribe`**（条件订阅）**。
- **原生支持“死区与节流订阅（Deadband & Throttling Subscription）”**：客户端（如 Home Assistant 或云端能源看板）向电表 `Pro 3EM Grid` 发起 `Subscribe` 时，可以直接在参数中指定：`{"min_interval": 1, "max_interval": 60, "delta": {"power": 10.0}}`。这意味着：**只有当功率波动超过 10W 时设备才立即推送报文；如果功率稳定不变，每 60 秒才发一次心跳**。仅这一项改进，就能将家庭局域网和云端服务器的无效报文流量削减 **90% 以上**。

第三层：传输与安全层 —— 【用 CoAP (UDP) + COSE 对象加密，取代重型 TCP/TLS】

为了解决 MCU 的 TCP 连接数上限与配电箱群控延迟问题，将局域网主通信底座从 WebSocket (TCP) 迁移至 **CoAP over UDP（或 Zenoh-Pico）**：

1. **无连接高并发 + 原生组播（Multicast）**：
   - 基于 UDP 的 CoAP 不需要维持 TCP 连接状态。家里哪怕有 20 个客户端同时监听一台开关，开关也只需要 1 个 UDP 端口即可应付。
   - **毫秒级群控**：当用户按下门口的无线旋钮开关 `Sense Knob4` 触发“全屋关灯”时，无需中枢逐个给 30 个开关发 TCP 请求，只需在局域网/Thread 网内发出 **1 个 UDP 组播包（Multicast）**，30 个开关在 **5 毫秒内** 整齐划一地同时断开继电器，彻底消除传统 Wi-Fi 智能家居关灯时的“波浪感（Popcorn Effect）”。
2. **用 COSE（RFC 9052 对象级加密）取代 HTTPS/TLS 证书地狱**：
   - 在局域网内给每个 IP 地址配置 HTTPS 证书是不可能的（浏览器会弹出烦人的不安全警告），而跑完整的 TLS 握手每次要消耗单片机近 40KB 的宝贵 RAM。
   - 采用基于 CBOR 的 **COSE（CBOR Object Signing and Encryption）** 或 **Noise 协议**：不加密传输通道，而是直接用预共享密钥（PSK）或配网派生的会话密钥**对那十几字节的 CBOR 载荷本身进行 AEAD（如 ChaCha20-Poly1305 或 AES-CCM）硬件加速加密与防重放签名**。不仅 RAM 开销接近于零，而且这包加密数据无论经过 HTTP、UDP、蓝牙还是串口转发，全程都是端到端安全的。

四、 直观对比：同一条“打开 0 号开关”指令的报文演进

假设我们要调用设备打开 0 号继电器，并在 5 秒后自动翻转关闭：

- **现行 Shelly JSON-RPC 2.0（纯文本，96 字节）**：

  ```
  {"id":101,"src":"ha_server","method":"Switch.Set","params":{"id":0,"on":true,"toggle_after":5}}
  ```

- **重构后：开发者在浏览器或 Webhook 中调用（100% 保持兼容）**：依然可以直接发送上述 JSON，或者直接访问 `http://ip/rpc/Switch.Set?id=0&on=true&toggle_after=5`。

- **重构后：设备内部、Thread 网络、BLE 与中枢之间实际传输的 Packed CBOR（仅 13 字节，十六进制展示）**：

  ```
  A4 01 18 65 02 69 03 60 04 A3 00 00 01 F5 02 05
  ```

  (注：仅需 ***13 个字节****！无需建连、无需分片、无需字符串比对，MCU 收到 UDP 包后在几个微秒内即可通过数组下标直接定位到 C 语言函数指针并执行)。*

通过下方的交互式工具，你可以调整不同应用场景（如**纽扣电池 Thread 传感器、高频采样三相电表、第三方极客 Webhook 集成**）的权重优先级，直观对比 **JSON-RPC 2.0**、**Protobuf**、**Matter TLV** 与我们重构设计的 **CBOR-RPC 双轨架构** 在报文体积、MCU 内存占用与开发友好度上的量化表现：

Sources

1. CBOR vs. the Other Guys - The CBOR, dCBOR, and Gordian
2. Modern Alternatives to JSON in Microservices: What Developers Are
3. Go Protobuf: The New Opaque API - Hacker News

User prompt: 解释下面的文字 第二层：数据模型与语义层 —— 【用“属性树 + 增量订阅”取代孤立的“过程调用”】 抛弃原本零散的 <Component>.<Method> 命名，在固件内部直接采用与 Matter Interaction Model（交互模型）同源的四元原语： - Read（读属性） / Write（写属性） / Invoke（执行动作） / Subscribe（条件订阅）。

Response: 这段话描述的是物联网（IoT）软件架构从“传统的面向自定义函数调用（RPC 自由方言）”**向**“现代面向标准数据模型与数字孪生（Data-Centric）”演进的质变过程。

要透彻理解这段话，可以将其拆解为三个核心问题：**过去的做法有什么硬伤？什么是“属性树”？为什么“四元原语”能够彻底重构系统交互？**

一、 过去的做法：什么是“零散的 `<Component>.<Method>` 过程调用”？

在 Shelly 现行的 Gen2/Gen3 架构（以及绝大多数传统 IoT 平台）中，设备的能力被抽象为一个个**独立的私有函数**。

比如，你想让设备打开开关、获取当前功耗、让设备重启，调用的是完全不同的自定义路径：

- 打开开关：调用 `Switch.Set`，传参 `{"id": 0, "on": true}`
- 读电量：调用 `Switch.GetStatus`，返回一堆嵌套 JSON
- 校准窗帘：调用 `Cover.Calibrate`
- 重启系统：调用 `Sys.Reboot`

这种模式在小规模时很直观，但在系统级架构中有三大硬伤：

1. **“方言林立”，无法跨厂商甚至跨品类复用**： 每个硬件厂商、甚至同一个厂商的不同工程师，设计的函数名各不相同。有的叫 `Switch.Set`，有的叫 `Relay.Toggle`，有的叫 `turn_on`。中枢系统（如 Home Assistant 或 Apple Home）每接入一个设备，都必须写一段专用的“胶水代码（Adapter）”去适配这些千奇百怪的函数名。
2. **状态与动作混为一谈**： 改变电灯的状态本质上只是将一个布尔变量从 `false` 改为 `true`，但在 RPC 体系下却被包装成了一次复杂的“远程函数调用（Procedure Call）”。
3. **固件端代码严重膨胀**： 单片机内部必须维护一张极其庞大的 `switch-case` 字符串分发路由表。如果设备支持 50 个方法，固件里就得写 50 个分支去解析不同的参数结构。

二、 破局核心：什么是“属性树（Attribute Tree）”？

属性树的思想源自工业现场总线（如 OPC UA）以及智能家居的全球统一标准 **Matter / Zigbee Cluster Library (ZCL)**。

它不再把设备看作“一堆可调用的函数集合”，而是把设备看作**一棵标准化的数据结构树（数字孪生模型）**。

示例：一个双路计量开关在“属性树”模型下的抽象

```
Device（设备根节点）
├─── Endpoint 0（系统公共端）
│    └─── BasicInformation Cluster
│         ├─── Attribute: SoftwareVersion = "1.0.4"
│         └─── Attribute: RebootCount = 12
├─── Endpoint 1（第 1 路开关）
│    ├─── OnOff Cluster
│    │    └─── Attribute: on_off = true (布尔值)
│    └─── ElectricalMeasurement Cluster
│         ├─── Attribute: active_power = 45.2 (瓦特)
│         └─── Attribute: rms_voltage = 228.5 (伏特)
└─── Endpoint 2（第 2 路开关）
     └─── OnOff Cluster
          └─── Attribute: on_off = false (布尔值)
```

在这个模型中，设备的任何硬件状态（开关、电压、亮度、温度），在内存中都对应着树上的一个明确路径（如 `Endpoint 1 / OnOff / on_off`）。

三、 交互基石：Matter 的“四元原语（Four Primitives）”

既然所有状态都变成了一棵“树”，那么外界与设备之间的所有交互动作，在数学上就可以收敛为**仅需 4 个标准的底层通用动词（原语）**：

原语	对应动词	含义与实际场景	为什么它比传统 RPC 优雅?
`Read`	**读属性**	读取树上的一个或多个节点。 *例：读取 Endpoint 1 下的所有电压和功率属性。*	支持**通配符读取（Wildcard Read）**。你可以发一条指令“读取全设备所有 Endpoint 的 `on_off` 属性”，一次调用就能拉取整屋所有回路的状态，不需要挨个调不同的 Get 方法。
`Write`	**写属性**	修改树上某个可写节点的值。 *例：将 Endpoint 1 的 `on_off` 节点写入 `true`。*	**状态驱动（State-Driven）**。控制开关不再是发“开命令”，而是把开关的目标状态改成“开”。单片机底层监听该变量变化，自动闭合物理继电器。接口极其干净。
`Invoke`	**执行动作**	触发一个非单纯修改数值的瞬态动作或流程。 *例：`Sys.Reboot()`、`Cover.Calibrate()` (校准窗帘行程)、`DoorLock.UnlockWithPin("1234")`。*	严格限制其作用域。只有当某个行为**无法通过修改属性来表达**（具有时间序列性、涉及临时密码鉴权或一次性瞬态过程）时，才走 `Invoke`。这使得 API 职责边界极为清晰。
`Subscribe`	**条件订阅**	**最核心的杀手锏**：客户端向设备注册订阅，设备根据设定的条件主动推送状态变化。	彻底取代了“客户端死循环轮询”和“设备无脑频繁全量广播”。

四、 灵魂所在：为什么“增量订阅（Delta Subscription）”是革命性的?

在传统基于 RPC 的智能家居中，设备的状态同步非常笨拙：

- **要么靠轮询**：中枢每隔 2 秒调一次 `Switch.GetStatus`。99% 的请求返回的数据完全一样，浪费网络带宽与 CPU。
- **要么无脑推全量**：开关一有风吹草动，设备就把包含几百字节的完整状态 JSON 包往 WebSocket 或 MQTT 里吐一遍。

而在“属性树 + 增量订阅”体系下，客户端在发起 `Subscribe` 时，可以附带精细的**滤波规则（Deadband & Throttling）**：

> **客户端发出订阅请求：** *“我订阅你的 `active_power`（有功功率）属性。规则：只有当功率变化幅度超过 **5W**，或者距离上次上报已经过去了 **60秒**（心跳保活），你才需要推送；否则哪怕每秒都在采样，你也保持沉默。”*

这带来的收益是颠覆性的：

1. **网络报文削减 90% 以上**：电网电压从 229.1V 微小波动到 229.2V 时，设备在底层默默过滤，不产生任何网络流量。
2. **只推增量（Delta），不推全量**：当且仅当变化触发时，设备只发出一小串极简的二进制/JSON 增量：`{"Endpoint": 1, "Cluster": "ElectricalMeasurement", "power": 65.5}` 单包数据量从原本的上百字节骤降到十几个字节。
3. **极大降低纽扣电池设备的功耗**：传感器不需要因为微小的环境噪声频繁唤醒射频芯片，使得电池寿命直接从 1 年延长到 3~5 年。

总结

这句话的核心含义是： 通过抛弃过去随心所欲命名的 `Switch.Set`、`Cover.Open` 等私有函数，让设备在底层回归为一棵标准透明的“状态数据树”。

外界对设备的一切操作被高度归纳为 **读（Read）、写（Write）、执行（Invoke）、订阅（Subscribe）** 4 个通用标准动作。这不仅让设备天然与 Matter 等国际下一代标准无缝接轨，而且借助**增量条件订阅**，以极低的带宽和计算资源开销，实现了高实时、低延迟的状态同步。

User prompt: 请用具体的数据帧格式（比如 JSON 或 CBOR），演示 Read、Write、Invoke 和带有死区滤波的 Subscribe 报文是如何具体交互的。

Response: 以下以一个具备**电能计量与双路继电器**的智能设备（如 Core 2PM）为例，演示 **Read、Write、Invoke** 以及带**死区滤波（Deadband）的 Subscribe** 在通信时的具体数据帧交互格式。

每个原语均采用统一的路径寻址：[Endpoint, Cluster, Attribute/Command]。示例以**直观的 JSON 格式**呈现，文末附上对应在底层传输时的 **Packed CBOR 十六进制压缩帧** 对比。

1. Read（读取属性）

客户端主动向设备查询指定路径或通配路径下的数据。

场景：读取 1 号通道（Endpoint 1）的当前电能计量数据（有功功率、有效电压）

- **客户端请求帧（Client → Device）：**

```
{
  "id": 201,
  "action": "Read",
  "paths": [
    {
      "endpoint": 1,
      "cluster": "ElectricalMeasurement",
      "attributes": ["active_power", "rms_voltage"]
    }
  ]
}
```

- **设备响应帧（Device → Client）：**

```
{
  "id": 201,
  "status": "Success",
  "data": [
    {
      "endpoint": 1,
      "cluster": "ElectricalMeasurement",
      "values": {
        "active_power": 62.4,
        "rms_voltage": 230.1
      }
    }
  ]
}
```

通配符扩展：若将 endpoint 设为 *，可单次批量拉取全设备所有通道的状态，无需逐个通道发送独立请求。

2. Write（修改属性）

面向状态的控制。通过将属性树上的变量置为目标值，由设备底层状态机联动硬件外设。

场景：将 1 号通道的开关状态置为打开（true）

- **客户端请求帧（Client → Device）：**

```
{
  "id": 202,
  "action": "Write",
  "modifications": [
    {
      "endpoint": 1,
      "cluster": "OnOff",
      "attributes": {
        "on_off": true
      }
    }
  ]
}
```

- **设备响应帧（Device → Client）：**

```
{
  "id": 202,
  "status": "Success",
  "results": [
    {
      "endpoint": 1,
      "cluster": "OnOff",
      "attribute": "on_off",
      "code": 0
```

```
    }
  ]
}
```

3. Invoke（执行动作）

针对**不可通过单纯数值覆写表达**的瞬态过程、带鉴权动作或复杂运行序列。

场景：触发窗帘电机的行程自动校准程序（带校准力矩参数）

- **客户端请求帧（Client → Device）：**

```
{
  "id": 203,
  "action": "Invoke",
  "endpoint": 1,
  "cluster": "WindowCovering",
  "command": "Calibrate",
  "args": {
    "torque_limit_pct": 80,
    "direction_reverse": false
  }
}
```

- **设备响应帧（Device → Client）：**

```
{
  "id": 203,
  "status": "Success",
  "command_response": {
    "state": "Calibrating",
    "estimated_duration_sec": 25
  }
}
```

4. Subscribe（条件增量订阅与死区滤波）

这是重构架构的核心交互：客户端向设备注册推送规则，设备只在**满足死区滤波条件**或**到达心跳时间**时主动发送增量。

场景订阅规则：

- 监听 1 号通道的 `active_power`（功率）。
- `min_interval: 1`：防抖限制，两次推送间隔至少 1 秒。
- `max_interval: 60`：心跳保活，若 60 秒内无变化，强制推送一次当前值。
- `delta: 5.0`：死区阈值。功率波动绝对值 $\geq 5.0\ \mathrm{W}$ 时触发推送。

Step 1: 建立订阅会话

- **客户端订阅请求（Client → Device）：**

```
{
  "id": 204,
  "action": "Subscribe",
  "criteria": {
    "min_interval": 1,
    "max_interval": 60,
    "deadband": {
      "endpoint": 1,
      "cluster": "ElectricalMeasurement",
      "attribute": "active_power",
      "delta": 5.0
    }
  }
}
```

- **设备确认并返回初始基准数据（Device → Client）：**

```
{
  "id": 204,
  "status": "Success",
  "subscription_id": "sub_8f2a9",
  "initial_report": {
    "endpoint": 1,
    "cluster": "ElectricalMeasurement",
    "values": {
```

```
      "active_power": 40.0
    }
  }
}
```

Step 2: 采样微小波动（处于死区内 → 设备静默）

- **物理环境变化**：设备内部传感器采样到功率发生微小波动：
  - $t = 2\text{s}$：$40.3\ \text{W}$（变化 $+0.3\ \text{W} < 5.0\ \text{W}$）
  - $t = 5\text{s}$：$39.8\ \text{W}$（变化 $-0.2\ \text{W} < 5.0\ \text{W}$）
  - $t = 9\text{s}$：$42.1\ \text{W}$（变化 $+2.1\ \text{W} < 5.0\ \text{W}$）
- **网络通信行为**：**0 报文传输**。设备内核直接在硬件过滤层拦截，网络接口与射频保持静默休眠，不产生任何网络流量。

Step 3: 越过死区阈值（触发增量上报 Delta Report）

- **物理环境变化**：$t = 12\text{s}$ 时，用户开启大功率电器，功率骤升至 $65.5\ \text{W}$（变化 $|65.5 - 40.0| = 25.5\ \text{W} \geq 5.0\ \text{W}$）。
- **设备主动推送增量帧（Device → Client）：**

```
{
  "subscription_id": "sub_8f2a9",
  "action": "Report",
  "timestamp": 1774338242,
  "delta": [
    {
      "endpoint": 1,
      "cluster": "ElectricalMeasurement",
      "values": {
        "active_power": 65.5
      }
    }
  ]
}
```

设备内部将基准值更新为 $65.5\ \text{W}$，后续判定基于该新值计算 Δ。

Step 4: 达到最大保活间隔（触发心跳保活 Heartbeat Report）

- **物理环境变化**：电器持续稳定运行，在随后的 60 秒内功率一直在 $65.0\ \text{W} \sim 66.0\ \text{W}$ 之间微弱浮动，未触发 $5.0\ \text{W}$ 差值。
- **网络通信行为**：$t = 72\text{s}$（距上次推送满 60 秒，触发 `max_interval`）。
- **设备主动推送保活报文（Device → Client）：**

```
{
  "subscription_id": "sub_8f2a9",
  "action": "Report",
  "heartbeat": true,
  "timestamp": 1774338302,
  "delta": [
    {
      "endpoint": 1,
      "cluster": "ElectricalMeasurement",
      "values": {
        "active_power": 65.2
      }
    }
  ]
}
```

5. 线上传输形态对比：JSON 与 Packed CBOR

上述 `Write` 指令（开灯）如果要在低功耗 Thread（802.15.4）网络或蓝牙 GATT 中传输，两者的载荷体积对比如下：

文本 JSON 形式（未压缩时 121 字节）

```
{"id":202,"action":"Write","modifications":[{"endpoint":1,"cluster":"OnOff","attributes":{"on_off":true}}]}
```

Packed CBOR 二进制形式（仅 16 字节）

通过将四元原语（`Write = 2`）与 Cluster/Attribute 映射为固定整型 Tag，二进制帧可以直接压缩至：

```
A4 01 18 CA 02 02 03 81 A3 01 01 02 06 03 A1 01 F5
```

- **字节拆解解析：**
  - `A4`：包含 4 个键值对的 Map
  - `01 18 CA`：id: 202
  - `02 02`：action: Write（整数标签化）
  - `03 81 A3`：modifications 数组
  - `01 01`：endpoint: 1
  - `02 06`：cluster: OnOff（集群 ID 0x0006）
  - `03 A1 01 F5`：attributes: { on_off (Tag 1): true (0xF5) }

该 16 字节载荷可直接置入单个 802.15.4 物理帧（安全上限约 88 字节），无需 6LoWPAN 分片，单次通信在几个微秒内即可完成硬件寄存器写入。

User prompt: 展示在嵌入式 C 语言中，设备端实现 deadband（死区滤波）与心跳周期的核心代码结构。

Response: 在嵌入式系统（如 ESP32、STM32 或 Nordic nRF）中，死区过滤与心跳保活的核心在于：**避免动态内存分配（零堆开销）、防止毫秒计时器溢出（Tick Wraparound）以及极低的 CPU 唤醒周期**。

以下是设备端实现该逻辑的典型 C 语言核心结构与状态机：

```c
#include <stdint.h>
#include <stdbool.h>
#include <math.h>

/* 上报原因枚举 */
typedef enum {
    REPORT_NONE = 0,
    REPORT_TRIGGER_DELTA,     /* 越过死区阈值上报 */
    REPORT_TRIGGER_HEARTBEAT  /* 达到最大间隔心跳上报 */
} report_trigger_t;

/* 单个属性的订阅过滤规则与运行时状态 (仅占用 24 字节 RAM) */
typedef struct {
    /* 静态配置参数 (来自客户端 Subscribe 报文) */
    uint32_t min_interval_ms;  /* 最小节流/防抖时间 (例如 1000 ms) */
    uint32_t max_interval_ms;  /* 最大保活心跳时间 (例如 60000 ms) */
    float    deadband_delta;   /* 死区阈值 (例如 5.0 W) */

    /* 运行时状态变量 */
    float    last_reported_val;     /* 上次成功推送时的基准值 */
    uint32_t last_reported_time_ms; /* 上次推送时的时间戳 (ms) */
    bool     has_initial_report;    /* 是否已完成首次上报标记 */
} deadband_filter_t;

/**
 * @brief  死区过滤评估核心函数
 * @param  filter: 过滤器实例指针
 * @param  current_val: 传感器本次采集到的物理量 (如当前功率)
 * @param  now_ms: 系统当前滴答时间 (如 esp_timer_get_time() / 1000)
 * @return report_trigger_t: 触发上报的原因
 */
report_trigger_t deadband_evaluate(deadband_filter_t *filter, float current_val, uint32_t now_ms)
{
    /* 1. 首次上报：订阅建立后的第一次采样必须无条件上报，确立初始基准 */
    if (!filter->has_initial_report) {
        filter->last_reported_val = current_val;
        filter->last_reported_time_ms = now_ms;
        filter->has_initial_report = true;
        return REPORT_TRIGGER_DELTA;
    }

    /* 2. 计算距离上次上报经过的时间 (无符号减法，天然免疫 uint32_t 溢出回绕) */
    uint32_t elapsed_ms = now_ms - filter->last_reported_time_ms;

    /* 3. 节流防抖 (Throttling)：未达到最小上报间隔，强行抑制 */
    if (elapsed_ms < filter->min_interval_ms) {
        return REPORT_NONE;
    }

    /* 4. 死区判断：波动差值绝对值 >= deadband_delta */
```

```
    if (fabsf(current_val - filter->last_reported_val) >= filter->deadband_delta) {
        filter->last_reported_val = current_val;
        filter->last_reported_time_ms = now_ms;
        return REPORT_TRIGGER_DELTA;
    }

    /* 5. 心跳保活 (Heartbeat)：变化在死区内，但已达最大等待时限 */
    if (elapsed_ms >= filter->max_interval_ms) {
        filter->last_reported_val = current_val;
        filter->last_reported_time_ms = now_ms;
        return REPORT_TRIGGER_HEARTBEAT;
    }

    return REPORT_NONE;
}
```

硬件事件主循环中的集成示例

在实际固件中，采样中断或周期任务通常以高频（如 100 ms）轮询硬件寄存器，通过上述状态机拦截无用数据：

```
/* 假设实例化一个针对“有功功率”的过滤器 */
static deadband_filter_t power_filter = {
    .min_interval_ms = 1000,   /* 至少间隔 1 秒 */
    .max_interval_ms = 60000,  /* 至多 60 秒必须报一次心跳 */
    .deadband_delta  = 5.0f,   /* 功率波动 >= 5W 触发 */
    .has_initial_report = false
};

void sensor_polling_task(void)
{
    /* 1. 物理层高速采样 (底层 ADC 或计量芯片计量值) */
    float current_power = read_meter_active_power();
    uint32_t now = get_system_uptime_ms();

    /* 2. 进入死区评估 */
    report_trigger_t trigger = deadband_evaluate(&power_filter, current_power, now);

    /* 3. 根据评估决策是否启动射频发送 */
    switch (trigger) {
        case REPORT_TRIGGER_DELTA:
            /* 组装 CBOR/JSON 差异帧，向已订阅的通道推送数据 */
            emit_subscription_report(ENDPOINT_1, CLUSTER_ELEC_MEAS, current_power, false);
            break;

        case REPORT_TRIGGER_HEARTBEAT:
            /* 组装心跳包推送，标识 heartbeat = true */
            emit_subscription_report(ENDPOINT_1, CLUSTER_ELEC_MEAS, current_power, true);
            break;

        case REPORT_NONE:
        default:
            /* 保持静默，无线射频无需唤醒发包，CPU 直接进入低功耗待机 */
            break;
    }
}
```

嵌入式工程落地中的设计关键点

1. **Tick 溢出安全性（Wraparound Safety）** `uint32_t` 毫秒计时器在约 **49.7 天**后会从 `0xFFFFFFFF` 溢出回转到 `0`。代码中使用 `(uint32_t)(now_ms - filter->last_reported_time_ms)` 利用了无符号整数的回绕补码特性，即使发生溢出，差值计算依然绝对正确。
2. **零堆分配与极小内存常驻** 每个属性过滤器的上下文仅需 24 字节。哪怕一个多路电表有 32 个监控属性，总内存占用也不足 1 KB，完全可以以内联数组形式固化在设备的数据模型树节点上。
3. **支持浮点与定点数（Fixed-Point）切换** 在无硬件浮点单元（FPU）的低端 Cortex-M0+ 或极低功耗 RISC-V 芯片上，可将 `float` 直接替换为定点整数（例如以毫瓦为单位的 `int32_t power_mw`，差值判断改用 `labs()`），单次评估仅需十几个汇编指令周期。

User prompt: - 如果在 固件内部直接采用与 Matter Interaction Model（交互模型）同源的四元原语 Read / Write / Invoke / Subscribe。在边缘转化到 JSON RPC 2.0 时，会不会对应不上？

Response: 你的直觉非常敏锐——这正是物联网架构师在将 **Matter 交互模型（四元原语）** 与 **JSON-RPC 2.0** 桥接时，最核心的工程难点。

如果只是做机械的“字面一比一直译”，**确实会在 4 个地方出现严重的“对应不上”（协议语义冲突）**。但在成熟的网关与固件设计中，这 4 个冲突可以通过一套极其精巧的 **“双模路由映射 + JSON-RPC 通知扩展”** 机制 100% 完美化解，甚至还能顺手把 Shelly 原本割裂的 Webhook 功能彻底统一起来。

以下是这 4 个潜在冲突点及其在边缘转译层的具体解决机制：

冲突一：`Subscribe` 的“一对多持续推送” vs JSON-RPC 的“一问一答”

- **为什么会对应不上？** JSON-RPC 2.0 规范严格要求：**1 个带 `id` 的 Request，只能对应 1 个相同 `id` 的 Response**，一旦返回响应，这次调用就结束了。 而四元原语中的 `Subscribe`（订阅）是一个**长生命周期会话**——客户端发起 1 次订阅后，设备在未来几小时内可能会陆陆续续主动推送成百上千次 `Report`（增量状态包）。此外，如果用户是通过无状态的 **HTTP POST** 发起 JSON-RPC，连接断开后设备往哪推数据？
- **边缘转译解决方案：借鉴以太坊（Ethereum JSON-RPC）与 LSP 的标准三步法 + Webhook 统一**

  1. **在长连接通道上（WebSocket / MQTT / 串口）—— 拆分为“1 次建连响应 + N 次无 ID 通知（Notification）”：**
     - **第 1 步（请求）**：客户端发送标准 JSON-RPC 请求（带 `"id": 101`）。

       `{"jsonrpc": "2.0", "id": 101, "method": "IM.Subscribe", "params": {"ep": 1, "cluster": "ElecMeas", "attr": "power`

     - **第 2 步（单次响应）**：设备立即回复一条包含 `sub_id`（订阅句柄）的标准响应，**合法终结 `id: 101` 的生命周期**：

       `{"jsonrpc": "2.0", "id": 101, "result": {"sub_id": "sub_01", "initial_val": 40.0}}`

     - **第 3 步（持续推送）**：后续每当越过死区触发上报时，转译层将其包装为 **JSON-RPC 2.0 标准 Notification（不带 `id` 字段）** 发送：

       `{"jsonrpc": "2.0", "method": "IM.Report", "params": {"sub_id": "sub_01", "ep": 1, "power": 65.5}}`

  2. **在无状态通道上（HTTP）—— 自动蜕变为 SSE 流或 Webhook！**
     - 如果开发者通过 HTTP 调用 `IM.Subscribe`，并在 `params` 里多传了一个 `"callback_url": "[http://192.168.1.10:8123/api/webhook](http://192.168.1.10:8123/api/webhook)"`，设备内部的死区滤波器一旦触发，就会自动向该 URL 发起 HTTP POST 推送。
     - **架构惊喜**：在旧版 Shelly 中，“WebSocket 状态推送”和“Webhook 动作触发”是两套完全独立的代码；而在新架构下，**Webhook 本质上就是一个指定了 HTTP 回调地址的 `Subscribe` 原语**，固件里直接少写了一半逻辑！

冲突二：树状寻址 `[ 原语 + Endpoint + Cluster + Attribute ]` vs 扁平的 `"method"` 字符串

- **为什么会对应不上？** 四元原语是**四维结构**（动作 + 端点 + 功能簇 + 属性名），而 JSON-RPC 2.0 只有一个扁平的字符串字段 `"method"`。如果强行让老用户写完整的树状路径，原来的极客习惯的 `Switch.Set?id=0&on=true` 就不能用了。
- **边缘转译解决方案：内置 2KB 的“静态别名映射表（ROM Alias Table）”实现双语法兼容**

  转译器在解析 JSON-RPC 的 `"method"` 时，通过前缀判断支持两种调用模式：

  1. **原生四元原语模式（面向现代插件，如 Home Assistant 新集成）：** `"method"` 直接写 `"IM.Read"`、`"IM.Write"`、`"IM.Invoke"`、`"IM.Subscribe"`，树状路径放在 `"params"` 中，实现 **100% 无损零开销映射**。
  2. **语法糖兼容模式（面向老用户 / 简易 `curl` / 旧脚本）：** 当转译器发现 `"method"` 不是以 `IM.` 开头（例如收到 `"method": "Switch.Set"`），它会在 Flash 常量区查询一张极小的静态映射表，将其**在内存里直接改写为四元原语**：

用户发来的传统 JSON-RPC 调用	边缘转译器自动映射成的内部四元原语
`Switch.GetStatus {"id": 0}`	→ `Read (ep = 1, cluster = [OnOff, ElecMeas], attr = *)`
`Switch.Set {"id": 0, "on": true}`	→ `Write (ep = 1, cluster = OnOff, attr = on_off, val = true)`
`Switch.Toggle {"id": 0}`	→ `Invoke (ep = 1, cluster = OnOff, cmd = Toggle)`
`Cover.Calibrate {"id": 0}`	→ `Invoke (ep = 1, cluster = WindowCovering, cmd = Calibrate)`
`Sys.Reboot {}`	→ `Invoke (ep = 0, cluster = BasicInfo, cmd = Reboot)`

(注：旧版 Shelly 的通道编号 `id: 0` 从 0 开始，而 Matter 标准 `Endpoint` 从 1 开始，因为 `Endpoint 0` 固定代表系统根节点——转译器会自动执行 `ep = id + 1` 的偏移转换)。

冲突三：“修改状态（Write）”与“带参动作（Invoke）”混合的模糊地带

- **为什么会对应不上？** 这是最容易踩坑的地方。在旧版 Shelly JSON-RPC 中，同一个 `Switch.Set` 方法，既可以只传 `{"id": 0, "on": true}`（单纯改状态），也可以传 `{"id": 0, "on": true, "toggle_after": 5}`（打开开关，但启动一个 5 秒倒计时自动关闭）。 在四元原语中，`Write` 只能修改静态属性（把 `on_off` 改成 `true`），不能带一个临时的运行参数 `toggle_after`！这该怎么对应？

- **边缘转译解决方案：利用 Matter 标准簇中的 `OnWithTimedOff` 命令进行条件分流**

  实际上，Matter 的 `OnOff Cluster` 早就考虑到了这种场景：它既有代表当前状态的 `on_off` **属性（Attribute）**，又内置了一个名为 `OnWithTimedOff(OnTime = 5s)` 的 **命令（Command）**。 因此，边缘转译器的规则非常清晰：

  - 如果收到 `Switch.Set` **仅包含** `"on": true/false` → 转译为 `Write` 属性 `on_off`（幂等状态覆写）。
  - 如果收到 `Switch.Set` **包含瞬态控制参数** `"toggle_after": 5` → 自动升级转译为 `Invoke` 命令 `OnWithTimedOff(duration=50)`（触发带定时器的动作）。

冲突四：批量操作的“部分成功（Partial Success）” vs JSON-RPC 的“全有或全无报错”

- **为什么会对应不上？** 在 Matter 的 `Write` 原语中，允许一次性修改 3 个属性（比如同时修改亮度、色温、开关状态）。如果其中 2 个成功了，但第 3 个因为数值超限失败了，四元原语会返回一个包含 3 个独立状态码的数组：`[Success, Success, ConstraintError]`。 但 JSON-RPC 2.0 规范规定：**响应报文里要么只有 `"result"`（代表成功），要么只有 `"error"`（代表失败），两者绝对不能共存**！
- **边缘转译解决方案：严格区分“协议层错误”与“业务层属性状态码”**

  1. **什么时候返回 JSON-RPC `"error"`？**
     - 只有当**整个请求在协议层或单目标操作完全失败**时（例如：JSON 语法错误、鉴权失败、或者只写 1 个属性但该属性报错），直接映射为标准 JSON-RPC `"error": {"code": -32602, "message": "ConstraintError"}`。
  2. **什么时候返回 JSON-RPC `"result"`？**
     - 当客户端发起的是**批量多属性** `IM.Write` 时，只要请求格式合法并成功送达内核，JSON-RPC 层面一律返回 `"result"`，并在 `"result"` 数组内部携带每个属性各自的状态码：

```
{
  "jsonrpc": "2.0",
  "id": 205,
  "result": {
    "has_failures": true,
    "results": [
      {"ep": 1, "attr": "on_off", "status": 0},
      {"ep": 1, "attr": "current_level", "status": 135, "error_msg": "OutOfRange"}
    ]
  }
}
```

五、 总结：C 语言边缘转译器的极简代码原型

为了证明这套转译不会给单片机带来负担，下面展示在固件 HTTP/WebSocket 边缘层，将 JSON-RPC 2.0 瞬时投射到内部四元原语的核心分发结构（仅需不到 100 行 C 代码）：

```
/* 静态 ROM 别名映射表：将传统 Shelly RPC 方法直接映射为四元原语路径 */
typedef struct {
    const char    *legacy_method;  /* 例如 "Switch.Set" */
    im_primitive_t primitive;      /* IM_READ / IM_WRITE / IM_INVOKE */
    uint16_t       cluster_id;     /* Matter 标准 Cluster ID */
    uint16_t       target_id;      /* Attribute ID 或 Command ID */
} rpc_alias_map_t;

/* 存放在 Flash 中，占用 0 字节 RAM */
static const rpc_alias_map_t k_legacy_aliases[] = {
    { "Switch.Set",       IM_WRITE,  CLUSTER_ON_OFF,           ATTR_ON_OFF          },
    { "Switch.Toggle",    IM_INVOKE, CLUSTER_ON_OFF,           CMD_TOGGLE           },
    { "Switch.GetStatus", IM_READ,   CLUSTER_ON_OFF,           ATTR_WILDCARD        },
    { "Cover.Open",       IM_INVOKE, CLUSTER_WINDOW_COVERING, CMD_UP_OR_OPEN       },
    { "Cover.Calibrate",  IM_INVOKE, CLUSTER_WINDOW_COVERING, CMD_CALIBRATE        },
    { "Sys.Reboot",       IM_INVOKE, CLUSTER_BASIC_INFO,      CMD_REBOOT           },
};

/* 当边缘收到一条 JSON-RPC 请求时的转译逻辑 */
void jsonrpc_edge_dispatch(const json_rpc_req_t *req, rpc_transport_t *transport)
{
    im_transaction_t im_tx = { .req_id = req->id };

    /* 分支 1：如果是原生四元原语调用 (如 "IM.Read", "IM.Subscribe") */
    if (strncmp(req->method, "IM.", 3) == 0) {
        parse_native_im_params(req->method + 3, req->params, &im_tx);
    }
    /* 分支 2：如果是传统 <Component>.<Method> 语法糖，查表自动转换 */
    else {
        const rpc_alias_map_t *alias = find_alias_in_rom(req->method);
        if (!alias) {
            send_jsonrpc_error(transport, req->id, -32601, "Method not found");
```

```
            return;
        }
        /* Endpoint 自动转换：legacy id (0-based) -> Matter Endpoint (1-based) */
        im_tx.endpoint   = json_get_int(req->params, "id", 0) + 1;
        im_tx.primitive  = alias->primitive;
        im_tx.cluster_id = alias->cluster_id;
        im_tx.target_id  = alias->target_id;

        /* 特殊兼容处理：如果 Switch.Set 带有 toggle_after 参数，自动升级为 Invoke */
        if (alias->cluster_id == CLUSTER_ON_OFF && json_has_key(req->params, "toggle_after")) {
            im_tx.primitive = IM_INVOKE;
            im_tx.target_id = CMD_ON_WITH_TIMED_OFF;
        }
        extract_im_payload_from_legacy_json(req->params, &im_tx);
    }

    /* 统一送入内核四元原语引擎执行！ */
    im_engine_execute(&im_tx, transport);
}
```

通过这一层轻量级的设计，设备在**内核层**享受到了 Matter 四元原语与 CBOR 的高内聚、低功耗与标准化红利；而在**外围表现层**，无论是遵循 JSON-RPC 2.0 规范的严谨客户端，还是习惯了 `Switch.Set` 的老用户，都能严丝合缝地顺畅对接。

User prompt: SHELLY GROUP 的产品如何保证安全性？在新架构下又如何保证安全性？

Response: 在智能家居与物联网后装模组市场中，由于设备直接串联在家庭 110V–240V 强电回路中，且暴露在局域网与无线空间内，其“安全性”必须同时涵盖**网络安全（Cybersecurity）**、系统与固件安全（System Reliability）**以及**物理电气安全（Electrical Safety）三个维度。

以下为您详细拆解 Shelly Group 现行产品的安全保障机制、存在的架构短板，以及在我们重构的新一代架构下如何实现全方位的安全升维。

一、 Shelly Group 现行产品（Gen2 / Gen3 / Gen4）是如何保证安全性的?

Shelly 能够在欧美主流市场通过严苛认证并被商业客户采纳，依靠的是以下四层现有安全防线：

1. 隐私与架构级安全：100% 本地闭环（Privacy by Design）

- **零云端强制暴露**：这是 Shelly 最大的安全底座。设备出厂即可彻底关闭 Shelly Cloud 连接，配合路由器防火墙禁止设备访问外网（WAN），所有控制流在局域网（LAN）内闭环。这从根本上免疫了“厂商云端服务器被黑客攻破导致全屋设备被远程接管”的行业系统性风险。
- **配网防蹭网保护**：新设备通电后的开放式 Wi-Fi AP 热点和蓝牙配网广播可设置超时自动关闭，或强制要求配网 PIN 码。

2. 局域网与无线通信安全：摘要认证与加密广播

- **HTTP / WebSocket 摘要认证（Digest Authentication）**： 在局域网内，当用户开启设备密码保护后，Shelly 的 JSON-RPC 并非明文传输密码，而是采用基于 **RFC 7616（SHA-256 Challenge-Response 质询-响应）** 机制。每次请求使用动态随机数（Nonce）与密码哈希计算签名，防止局域网抓包直接窃取管理员密码。
- **外发通道强制 TLS 1.2/1.3**： 当设备连接外部 MQTT Broker（MQTTs）、发送 HTTPS Webhook 或连接云端时，内置 mbedTLS 协议栈支持完整的 TLS 加密，并允许用户上传自定义的根证书（Custom CA Certificate）以对接企业内网私有服务器。
- **蓝牙传感器防重放加密（BTHome v2）**： 针对 Shelly BLU 蓝牙无线按钮和门磁，支持开启 **AES-CCM 对称加密**。每次按键广播不仅加密载荷，还携带严格递增的 32 位包计数器（Packet ID），防止攻击者用无线电设备录制广播包进行“重放攻击（Replay Attack）”偷开车库门。

3. 芯片与固件级安全：安全启动与签名校验

- **硬件级身份凭证（Matter DAC）**：在搭载自研定制芯片的 Gen3 / Gen4 设备上，出厂时在芯片一次性可编程区域（eFuse）烧录了符合 CSA 联盟规范的设备认证证书（DAC），防止假冒硬件混入生态。
- **固件数字签名校验**：OTA 升级包包含加密哈希与数字签名，设备在写入 Flash 闪存前会校验固件完整性，防止刷入被篡改的恶意固件（但同时保留了物理串口供极客主动刷写开源固件）。

4. 物理与电气安全：多重传感器与硬件熔断

- **三电保护（PM 计量型号）**：支持设置软件与硬件联动的**最大功率（Over-Power）、最大电流（Over-Current）、最高电压（Over-Voltage）阈值**。一旦电动汽车充电或取暖器短路超标，毫秒级自动断开继电器。
- **暗盒过温保护（Over-Temperature）**：芯片主板靠近继电器处内置 **NTC 热敏电阻**。当通风不良的暗盒内部温度超过安全阈值（通常为 95°C～100°C）时，设备强制切断负载输出并发送高温告警。
- **阻燃与安规**：外壳采用 **UL94-V0 级阻燃 PC 材质**，通过欧洲 VDE、ENEC 及北美 UL 安规认证。

二、 现行架构在安全性上的 4 大核心短板

尽管上述措施优于大多数廉价白牌产品，但受限于 Gen2/Gen3 的“纯文本 JSON-RPC over HTTP/TCP”及一体化硬件形态，现行体系存在 4 个难以根治的安全隐患：

1. **局域网报文“半裸奔”（无端到端载荷加密）：** 因为给局域网 IP 地址签发受浏览器信任的 HTTPS 证书极其困难（自签名证书会导致浏览器弹出刺眼的红色拦截警告，且每次 TLS 握手消耗近 40KB 单片机内存），Shelly 的局域网 HTTP/WebSocket 默认只能做 Digest 身份鉴权，**但 JSON-RPC 报文内容本身依然是明文传输的**。同一 Wi-Fi 下的恶意设备可以轻易监听全屋开关动作。
2. **“全有或全无”的粗放权限模型（单一 Admin 账号）：** 现行架构只有一个全局 `admin` 密码。一旦你把密码交给了某个第三方插件、租客或自动化脚本，它就拥有了设备的 **最高 Root 权限**——不仅可以开关灯，还能读取你的 Wi-Fi 密码、修改过流保护上限、甚至往设备里注入一段恶意的 mJS 脚本把设备变成局域网跳板机。
3. **本地 mJS 脚本缺乏硬隔离，存在“软变砖”风险：** 用户编写的本地 JavaScript 脚本与系统核心共享有限的内存池。如果脚本写了死循环、内存泄漏或高频网络请求，极易导致设备看门狗超时重启，甚至陷入“开机运行脚本 → 崩溃重启”的死循环，只能拆开墙壁面板长按物理复位键救砖。
4. **强弱电一体化带来的物理触电风险：** 普通用户在升级、更换或调试模块时，必须直接面对裸露的 220V 强电端子；且在窗帘电机等正反转场景中，若固件卡死，缺乏纯硬件电路级的物理互锁保障。

三、 重构后的新架构如何全方位保障并升维安全性？

在我们前文设计的 **“12 款模块化 SKU + CBOR/JSON 同构双轨 + Matter 四元原语 + CoAP/UDP”** 新架构下，可以从底层彻底填平上述 4 个安全坑洞，构建起工业级的 **5 层纵深防御体系**：

```
新架构五层纵深安全防御体系 (Defense-in-Depth)
├──── 1. 芯片硬件层：eFuse 硬件信任根 (RoT) + 硬件加密引擎 + 16MB A/B 双分区防变砖回滚
├──── 2. 传输加密层：COSE / OSCORE 对象级加密 (抛弃重型 TLS，16字节报文自带防重放 AEAD 装甲)
├──── 3. 语义权限层：基于属性树的细粒度 ACL (区分只读监控、普通开关控制、安全阈值与管理员)
├──── 4. 边缘沙盒层：受限脚本沙盒 (CPU时间片配额 + 独立静态内存池 + 连续崩溃自动熔断挂起)
└──── 5. 电气结构层：强弱电物理分离插拔 + 硬件过零磁保持 + 双路正反转纯硬件电路互锁
```

1. 传输与载荷安全：用 COSE / OSCORE 对象级加密终结“局域网裸奔”

在新架构中，设备间与中枢（Home Assistant / App / 局域网网关）通信不再依赖笨重且有证书烦恼的 HTTPS/TLS，而是采用 IETF 专为 CBOR 制定的 **COSE（RFC 9052，CBOR 对象签名与加密）** 与 **OSCORE（RFC 8613）**：

- **加密数据包本身，而非加密管道：** 每一条压缩后仅十几个字节的 CBOR 指令，在发出前直接调用芯片内置的硬件加密加速器（AES-CCM 或 ChaCha20-Poly1305）进行 **AEAD（带关联数据的认证加密）**。
- **天然抗重放攻击：** 每个加密对象头部自带 4 字节严格递增的序列号（Sequence Number）与会话标识。攻击者即便在局域网或 Thread 空中截获了数据包，既无法解密内容，也无法重放旧指令。
- **极低资源消耗：** 无需三次握手建连，无需分配 40KB 的 TLS 缓冲区，仅增加 **8～12 字节** 的认证标签（Tag）开销，就能让 UDP、HTTP、蓝牙、甚至串口上的每一条指令实现端到端加密。

2. 访问控制安全：基于“属性树与四元原语”的细粒度 ACL（权限分级）

得益于我们将固件内核重构为 `[Endpoint, Cluster, Attribute]` 属性树与 `Read / Write / Invoke / Subscribe` 四元原语，我们可以零成本引入与 Matter 标准完全对齐的 **ACL（Access Control List，访问控制列表）**，彻底告别“一个密码管所有”：

权限角色 (Role)	允许的原语动作	允许访问的属性树范围	典型授权对象与安全价值
`View` **(只读观察者)**	仅 `Read`, `Subscribe`	仅限功率、电压、温湿度等传感器 Cluster	授权给能源看板、租客或第三方统计插件，**绝无可能误触开关或修改配置**。
`Operate` **(日常控制者)**	`Read`, `Write`, `Subscribe`	仅限 `OnOff`、`LevelControl`、`WindowCovering` 等业务 Cluster	授权给家庭成员、普通墙壁面板或日常自动化，可开关灯，但**无法修改安全阈值或网络设置**。
`Manage` **(运维配置者)**	包含 `Invoke`	开放校准命令、脚本启停、本地规则表修改	授权给系统集成商或高阶自动化中枢。
`Admin` **(系统最高权限)**	全部四元原语	开放 `Endpoint 0`（系统配置、Wi-Fi/网络凭证、安全保护阈值、OTA 升级、ACL 分配）	仅限房东/业主本人持有，修改关键电气保护阈值（如过流上限）需二次校验。

3. 固件与边缘计算安全：16MB A/B 原子回滚 + 防变砖脚本沙盒

- **A/B 双分区原子级 OTA 与看门狗自动回滚**：因为新架构统一标配了 **16MB 大容量 Flash**，我们可以采用完整的 **A/B 双系统镜像分区**。升级新固件时写入静默的 B 分区，校验椭圆曲线签名（ECDSA P-256）通过后切换引导标志；如果新固件在开机后 **30 秒内** 未能成功完成硬件自检并喂狗（Watchdog），Bootloader 会在下一次重启时**毫秒级自动切回旧的 A 分区**，实现 100% 永不变砖。
- **带配额与熔断器的脚本安全沙盒（Sandboxed Runtime）**：
  - **资源硬隔离**：本地脚本分配固定大小的独立静态内存数组（如 32KB Arena），禁止直接访问系统底层网络栈和加密密钥区。
  - **CPU 时间片熔断**：单次事件回调执行时间超过 **50 毫秒**（如写出了 `while(true)` 死循环），硬件定时器立即强制中断该脚本并抛出异常，绝不阻塞物理开关面板响应和网络通信。
  - **三振出局安全模式（Safe-Mode Boot）**：如果设备在 60 秒内因异常连续非正常重启达 3 次，开机时自动挂起所有用户脚本与规则表，确保基础照明和 WebUI 救砖通道永远在线。

4. 物理与电气安全升维：强弱电分离 + 硬件级机电互锁

- **强弱电分离架构消除触电风险**：在模块化设计下，220V 强电端子、变压器与大功率继电器被永久封装在通过 VDE/UL 认证的暗盒底座内；用户日常插拔升级、按键复位或连接调试针脚，接触的只是外部 **3.3V/5V 安全特低电压（SELV）** 的弱电智能核心。
- **纯硬件电路级正反转互锁（Hardware Interlock）**：在 `Core 2PM` 和 `Pro 4PM Ultra` 的窗帘/卷帘门电机模式下，除了软件层面的互锁，还在继电器驱动电路上设计了 **硬件逻辑门互锁（Hardware XOR/NAND Interlock）**——即便单片机 CPU 被强电磁干扰跑飞，物理电路也绝对不可能同时导通“上升”和“下降”两路继电器，从物理层面杜绝烧毁昂贵窗帘电机的可能。
- **过零检测 + 磁保持继电器 + 插脚温控**：仅在交流电电压过零点切换触点，消除电弧拉火；吸合后线圈零电流零发热，使暗盒内部温升比传统 Wi-Fi 继电器降低 **15°C～20°C**，大幅降低老房线路老化起火风险。

四、 总结：现行架构 vs 新重构架构安全性全景对比

安全维度	Shelly 现行架构 (Gen2/Gen3/Gen4)	重构后的新一代架构方案	核心安全收益
局域网通信加密	仅头部 Digest 鉴权，JSON 载荷明文	**COSE / OSCORE 对象级 AEAD 加密**	彻底防窃听、防篡改、防重放，且免除 TLS 证书配置痛苦
权限与访问控制	单一全局 `admin` 密码（全有或全无）	**基于属性树的 4 级细粒度 ACL**	租客、看板、脚本权限严格隔离，防止越权修改安全阈值
本地脚本安全性	与系统共享内存，死循环易导致设备假死	**独立内存沙盒 + 50ms 超时中断 + 崩溃安全模式**	用户脚本随便折腾，物理开关与救砖后台永远不死机
固件升级可靠性	早期 4MB Flash 空间紧张，单区覆盖风险高	**16MB Flash 标配 A/B 双分区 + 看门狗原子回滚**	断电或固件 Bug 自动切回旧版本，实现“零变砖率”
电气与物理防护	强弱电一体；传统继电器持续发热；软件互锁	**强弱电分离插拔 + 过零磁保持 + 硬件电路互锁**	用户升级零触电风险，暗盒大幅降温，杜绝电机烧毁

User prompt: 如果要兼容现有的用户，是否无法解决下面这个问题？ 受限于 Gen2/Gen3 的“纯文本 JSON-RPC over HTTP/TCP”及一体化硬件形态，现行体系存在 4 个难以根治的安全隐患： 1. 局域网报文“半裸奔”（无端到端载荷加密）： 因为给局域网 IP 地址签发受浏览器信任的 HTTPS 证书极其困难（自签名证书会导致浏览器弹出刺眼的红色拦截警告，且每次 TLS 握手消耗近 40KB 单片机内存），Shelly 的局域网 HTTP/WebSocket 默认只能做 Digest 身份鉴权，但 JSON-RPC 报文内容本身依然是明文传输的。同一 Wi-Fi 下的恶意设备可以轻易监听全屋开关动作。

Response: **并非无法解决。**“兼容老用户的明文调用习惯”与“实现局域网端到端加密”看似非此即彼，实则可以通过**将“网页静态外壳传输”与“业务数据载荷加密”解耦**，配合**单端口多态协商**与**权限沙盒隔离**完美化解。

在不破坏现有用户任何一行旧脚本、不触发浏览器红色证书警告、且不增加单片机 40KB TLS 内存开销的前提下，工程上可以通过以下 **4 项关键技术** 彻底解决这一隐患：

一、 针对浏览器 WebUI：用“HTTP 传静态外壳 + 应用层 PAKE 加密”绕开证书死局

浏览器之所以在访问 `[https://192.168.1.50](https://192.168.1.50)` 时弹出刺眼的红色拦截警告，是因为国际 CA 机构严禁为 `192.168.x.x` 这类私有局域网 IP 签发受信任的 SSL 证书；而单片机之所以内存吃紧，是因为完整的 TLS 握手需要解析庞大的 X.509 证书链。

破局方法：放弃在传输层做 HTTPS，改在网页 JavaScript 应用层做端到端加密。

1. **第一步：静态网页外壳走普通 HTTP（零证书警告）** 用户在浏览器输入 `[http://192.168.1.50](http://192.168.1.50)` 时，设备依然通过普通 HTTP 返回那个 40KB 的单文件网页（HTML + JS）。因为这个静态文件里**只包含公开的界面代码，没有任何用户的隐私状态或密码**，所

以明文传输毫无泄密风险，且浏览器永远不会弹出任何证书错误警告。

2. **第二步：JS 在浏览器内建立应用层加密隧道（仅占 MCU 不到 2KB 内存）** 网页加载完成的瞬间，运行在浏览器里的 JavaScript 利用内置加密引擎，通过 WebSocket 与设备进行一次 **SPAKE2+（基于密码的认证密钥交换，也是 Matter 配网的标准算法）** 或 **X25519 密钥协商**：
    - 如果设备设了密码：双方无需在网络上传输密码，即可通过密码派生出完全相同的会话密钥（Session Key）。
    - 如果设备未设密码：采用 **TOFU（首次信任绑定）** 或临时椭圆曲线 ECDH 交换生成动态密钥。
3. **第三步：业务载荷全加密（COSE / ChaCha20-Poly1305）** 随后的所有 JSON-RPC / CBOR 状态推送与开关控制指令，全部由网页 JS 加密为二进制密文后再通过 WebSocket 发送。
    - **最终效果**：用户在地址栏依然像以前一样输入 `[http://192.168.1.50](http://192.168.1.50)` 即可秒开网页，但同一 Wi-Fi 下的窃听者抓包看到的WebSocket 载荷全部是无法破译的乱码；而单片机省去了庞大的 TLS 协议栈，仅需不到 **2KB RAM** 跑硬件对称加密。

*(注：若在极端高安全场景下还需防范黑客在局域网内通过 ARP 欺骗篡改首次加载的 JS 文件，可采用类似 Plex 媒体服务器的 `*.shelly.direct` **泛域名解析方案**——将 `192-168-1-50.<设备ID>.shelly.direct` 在公网 DNS 指向局域网 IP `192.168.1.50`，即可合法申请免费的 Let's Encrypt 泛域名 HTTPS 证书，配合 TLS 1.3 PSK 会话复用将内存压至 8KB 以内)。*

二、 针对智能家居中枢（HA / App / 新插件）：同一端口的“多态自动升级”

现有用户最怕的是“升级固件后，原来的接口地址变了，导致家里的自动化全挂了”。因此，设备必须保留原有的 `http://<IP>/rpc` 和 `ws://<IP>/rpc` 入口不变，采用 **HTTP 内容协商（Content Negotiation）** 实现新老同堂：

- **现代客户端自动走加密通道（占局域网流量的 95%）**： 像 Home Assistant 官方集成（`aioshelly`）、官方手机 App、Node-RED 插件等是由官方或开源社区统一维护的。只需更新插件版本，它们在访问 `/rpc` 时会自动带上请求头 `Content-Type: application/cose`（或在 WebSocket 握手时声明 `Sec-WebSocket-Protocol: shelly-cose`）。设备识别后，立即无缝切入 **COSE 二进制加密模式**。用户无需修改任何配置，家里的主力通信就已经在后台静默完成了加密升级。
- **老旧脚本依然识别明文 JSON（占局域网流量的 5%）**： 当设备检测到发往 `/rpc` 的请求头是传统的 `application/json` 或者是簡單的 `GET /rpc/Switch.Set?id=0&on=true` 时，自动调用上一轮提到的 3KB 边缘转译器处理。

三、 针对死活不改代码的旧脚本（`curl` / 旧 Webhook）：“明文降级沙盒”与权限熔断

你可能会问：**“只要还允许旧的 `curl` 发送明文 JSON，那这 5% 的兼容口子不还是半裸奔吗？”**

这就是新架构中 **“四元原语 + 细粒度 ACL（访问控制列表）”** 发挥决定性作用的地方。在旧架构中，为了一行 `curl` 开灯脚本兼容，你被迫把整台设备的最高管理员接口都暴露在明文下；而在新架构中，我们将明文请求严格关进 **“低权限隔离沙盒（Plaintext Quarantine）”**：

1. **高危操作强制加密（零妥协）**： 凡是涉及 `Endpoint 0`（修改 Wi-Fi 密码、读取系统配置、上传本地脚本、修改过流/过温安全阈值、OTA 升级、分配权限）的 `Admin` 和 `Manage` 级请求，**固件底层一刀切拒绝明文 JSON**，必须通过加密的 WebUI 或 COSE 通道才能执行。黑客就算在局域网抓包，也永远抓不到任何涉及系统安全的报文。
2. **普通开关操作提供“三级安全策略”供用户自选**： 针对仅涉及 `Operate`（开关灯）和 `View`（读功率）的旧版明文接口，系统提供平滑过渡机制：
    - **策略 A（白名单放行）**：允许明文 `curl` 调用 `Switch.Set`，但仅限来自指定局域网固定 IP（如用户 NAS 的 IP）的请求，其他设备一律拒绝。
    - **策略 B（免加密库的一行 URL 签名保护）**：如果老用户的简陋客户端（如老旧网络摄像机、简易单片机）装不下加密库，但想防篡改，支持在 URL 后追加标准 SHA-256 签名参数，而无需加密整个报文。
    - **策略 C（纯加密模式）**：当用户确认家里所有脚本都已升级后，可在后台一键关闭“Legacy 明文兼容开关”，实现全设备 100% 强制加密。

四、 总结：兼容性与安全性的全景兼顾效果

通过这套组合拳，我们在**不抛弃任何一个老用户**的前提下，彻底扭转了局域网的安全态势：

局域网访问来源	用户侧是否需要改代码/改习惯?	浏览器是否会弹红色证书警告?	实际在局域网空中传输的形态	安全性提升效果
浏览器直接访问 IP *(内置 WebUI)*	❌ **不需要** *(依然输入 `http://IP`)*	❌ **绝不弹窗** *(静态外壳走 HTTP)*	**应用层 AEAD 密文** *(JS 内置 SPAKE2+ 加密)*	**彻底根治窃听与篡改**，且单片机仅耗 2KB 内存

局域网访问来源	用户侧是否需要改代码/改习惯?	浏览器是否会弹红色证书警告?	实际在局域网空中传输的形态	安全性提升效果
Home Assistant / 官方 App *(主力高频交互)*	❌ **不需要** *(插件后台自动更新)*	不涉及	**COSE / OSCORE 二进制密文** *(单端口自动协商升级)*	**全屋 95% 以上高频状态与控制流实现端到端加密**
修改系统配置 / 上传脚本 *(高危管理操作)*	仅允许在 WebUI 或新版 CLI 中操作	❌ 不弹窗	**强制端到端加密**	**彻底封死黑客通过局域网接管设备或注入恶意脚本的路径**
十年前的老旧 `curl` 脚本 *(极客遗留 Webhook)*	❌ **不需要** *(原样兼容运行)*	不涉及	明文 JSON（被限制在 **低权限沙盒 + IP 白名单** 内）	就算被旁路嗅探，攻击者最多只能看到某盏灯亮灭，**无法越权触碰任何系统与安规设置**

User prompt: 通过 WebSocket 进行 OTA 升级，会不会把 F/W 编码，额外增加数据?

Response: 通过 WebSocket 进行 OTA 升级是否会编码固件并增加额外数据，**完全取决于你在 WebSocket 上使用的是“文本帧（JSON）”还是“二进制帧（Binary / CBOR）”**。

如果直接把固件塞进传统的纯文本 JSON-RPC 报文中传输，固件必须经过 Base64 编码，数据量会**白白暴增 33.3%**；但如果利用 WebSocket 原生支持的**二进制帧（Binary Frame）配合我们重构的 CBOR 架构（或信令与数据流分离架构）**，固件完全**不需要任何编码转换**，额外协议开销**低于 0.6%**。

以下是三种具体实现方式在底层字节流上的差异对比：

一、 糟糕的做法：把固件塞进纯文本 JSON-RPC（数据膨胀 33.3%）

在传统的纯文本 JSON-RPC 2.0 中，因为 JSON 字符串不支持直接包含不可见的原始二进制字节（如 `0x00`、换行符或未转义引号），开发者被迫将固件切片编码为 **Base64 字符串**（或 Hex 十六进制字符串）：

```
{
  "id": 501,
  "method": "OTA.WriteChunk",
  "params": {
    "offset": 4096,
    "data": "7f454c460101010000000000..."
  }
}
```

- **体积暴增**：Base64 编码会将每 3 个原始字节拆解映射为 4 个 ASCII 字符，导致 1 MB（1,024 KB）的固件包在网络上传输时直接膨胀到 **约 1.37 MB**（如果用 Hex 编码则会膨胀 **100%** 到 2 MB）。
- **单片机算力与内存双杀**：单片机收到这串庞大的 JSON 文本后，必须在极其有限的 RAM 中先分配一块缓冲区解析 JSON 字符串，再分配另一块缓冲区将 Base64 解码还原成二进制字节，最后才能写入 Flash 闪存。

二、 架构重构后的做法：WebSocket 二进制帧 + CBOR 原生字节数组（零编码，开销低于 0.6%）

很多人误以为 WebSocket 只能传文本，实际上 WebSocket 标准（RFC 6455）在协议头第 1 个字节就原生定义了两种模式：

- `Opcode = 0x1`（**Text Frame**）：传输 UTF-8 文本（如 JSON）。
- `Opcode = 0x2`（**Binary Frame**）：直接传输原始二进制字节流，**不做任何字符编码**。

在我们重构的 **CBOR-RPC / Matter 四元原语** 架构下，通过 WebSocket 二进制帧进行 OTA 升级的过程如下：

1. CBOR 原生支持裸二进制切片（Byte String）

CBOR 规范中的 `Major Type 2` 是专门的**原生字节数组（bstr）**。假设每次按照单片机 Flash 的一个扇区大小（**4,096 字节 / 4 KB**）发送固件切片，CBOR 只需在 4,096 个原始固件字节前面加上 **3 个字节的长度头**（`0x59 0x10 0x00`，代表后续紧跟 4,096 个原始字节），后面的固件内容原封不动直接拼接，**0% 编码膨胀**。

2. 精确计算单次 4 KB 固件切片的字节开销

当浏览器或手机通过 WebSocket 发送一个包含 4,096 字节固件切片的加密 CBOR 包时，线上的真实字节构成为：

`[ WebSocket 帧头: 8 Bytes ] + [ CBOR/COSE 指令与加密头: ~22 Bytes ] + [ 原始固件二进制数据: 4096 Bytes ]`

- **WebSocket 协议帧头（仅 8 字节）**：包含 1 字节操作码（`0x82` 代表二进制帧）、1 字节掩码标志与长度指示、2 字节扩展长度（表示 4 KB 长度）、4 字节客户端掩码密钥（Masking Key）。
- **CBOR 包装与 COSE 加密签名（约 22 字节）**：包含 `Invoke` 原语标签、块偏移量 `offset` 以及 16 字节的 AEAD 防篡改认证标签（Tag）。
- **总计额外开销**：每传输 4,096 字节的固件，仅增加约 **30 字节** 的包头，**额外数据比例仅为 0.73%**（若按 8 KB 切片传输，开销占比更是降至 **0.36%**）。
- **零拷贝直写 Flash**：单片机收到数据后无需做 Base64 解码，流式解密校验通过后，直接将缓冲区内的原始字节通过 SPI DMA 刷入 Flash 的 B 分区。

三、 兼容旧生态的做法：WebSocket 仅传控制信令 + HTTP 二进制流传输（Shelly 现行方案）

如果在保留纯文本 JSON-RPC 的前提下，又不想忍受 Base64 编码膨胀，Shelly 现行固件（Gen2/Gen3）采用的是 **“控制面与数据面分离（Out-of-Band）”** 的聪明做法：

1. **拉取模式（URL Pull）**： 客户端通过 WebSocket 发送一条只有几十字节的 JSON-RPC 指令：`{"id": 1, "method": "Shelly.Update", "params": {"url": "[http://192.168.1.10/firmware.bin](http://192.168.1.10/firmware.bin)"}}` 设备收到指令后，自己作为 HTTP Client 向该网址发起标准的 HTTP GET 请求，以纯二进制流（`application/octet-stream`）边下载边刷写 Flash。
2. **推送模式（HTTP POST Stream）**： 当用户在内置 WebUI 网页上点击“上传本地固件文件”时，网页并不会把固件塞进 WebSocket 的 JSON 里，而是单独向设备的 `http://<IP>/ota` 端口发起一次标准的二进制文件流推送，同时利用已有的 WebSocket 连接实时接收设备推送回来的升级进度百分比（如 `{"progress": 45}`）。

四、 三种 OTA 传输方案量化对比（以传输 1 MB 固件为例）

评估指标	方案 1：WebSocket 文本帧 + JSON (Base64)	方案 2：WebSocket 信令 + 独立 HTTP 二进制流 (Shelly 现行)	方案 3：WebSocket 二进制帧 + CBOR 原生字节流 (重构推荐)
是否需要编码固件	❌ 需要（Base64 编码）	✅ **不需要（原始二进制）**	✅ **不需要（原始二进制）**
1 MB 固件实际传输体积	**1,365 KB** (+33.3% 冗余)	**~1,026 KB** (+0.2% HTTP 头)	**~1,031 KB** (+0.7% 帧与加密头)
占用单片机 TCP 连接数	1 个（复用当前 WS 连接）	**2 个**（1 个 WS + 1 个额外 HTTP 连接）	**1 个**（完全复用当前 WS/CoAP 通道）
Thread / BLE 跨协议通用性	❌ 差（体积太大，低速网极慢）	❌ **不支持**（蓝牙/串口无独立 HTTP 通道）	✅ **极佳**（同一套分片逻辑通吃 WS、UDP、BLE 与串口）
单包防篡改与断点续传	✅ 支持按块校验与续传	⚠️ 整包流式传输，断线常需从头重传	✅ **每 4KB 独立 AEAD 签名，断线毫秒级原位续传**

总结： 只要在工程实现上避开“把二进制塞进 JSON 字符串”的误区，改用 **WebSocket 二进制帧**（`Opcode = 0x2`）**搭配 CBOR 原生字节流**（或者采用 **WebSocket 传信令 + HTTP 传二进制流**），通过 WebSocket 进行 OTA 升级就**完全不会对固件进行编码，也几乎不会增加额外的数据体积**。
